

=====

FILE: docs/README.md

=====

Welcome to Mintware

Mintware is a DeFi reputation and rewards platform built for EVM chains.

We turn your on-chain history into a verifiable score ? then use that score to determine how much you earn from campaigns, liquidity pools, and protocol incentives. The better your on-chain track record, the more your activity is worth.

Mintware is also designed to make on-chain actions easier to understand before you commit. Swaps, claims, and funding actions now surface clearer confirmation context, stronger chain guidance, and better fee visibility before your wallet opens.

Three Products, One Stack

Attribution ? On-Chain Reputation

A live scoring engine that analyses your full wallet history across 100+ chains and produces a single composite reputation score. Your score reflects the depth, consistency, and quality of your on-chain behaviour ? not just your balance.

Campaigns ? Earn from Your Activity

A rewards engine where protocols deposit tokens or points budgets, and Attribution score determines how large a share you earn. Three campaign types: Token Reward Pools, Points Campaigns, and AI Agent Campaigns.

Campaign and claim flows are designed to be clearer and safer, with better step guidance, fewer unnecessary approval prompts, and stronger transaction checks before submission.

Mintware Swap ? Clearer Before You Confirm

Mintware Swap is built to reduce confusion before the wallet popup appears.

Depending on the route and token pair, Mintware can show:

- what you are sending
- what you are expected to receive
- estimated network fees
- route and chain context
- warnings for missing gas balance or unusual price impact

Social Liquidity Vaults *(Coming Soon)*

LP vaults built on Uniswap V4 where your share of the fee pool is weighted by your Attribution score and referral network. The same deposit earns more if you've built a stronger on-chain reputation.

Vault flows are also being built with clearer staged guidance around approvals, deposits, and higher-trust actions.

Quick Links

- * [What is Attribution?](overview/attribution.md)
- * [How campaigns work](campaigns/overview.md)
- * [Token Reward Pools](campaigns/token-reward-pools.md)
- * [Points Campaigns](campaigns/points-campaigns.md)
- * [AI Agent Campaigns](campaigns/ai-campaigns.md)
- * [Social Vaults](vaults/overview.md)
- * [Connect your wallet](getting-started/connect-wallet.md)
- * [API reference](developers/api-reference.md)
- * [Smart contracts](developers/smart-contracts.md)

Platform Status

Product	Status
Attribution scoring	? Live ? 100+ chains
Token Reward Pools	? Live
Points Campaigns	? Live
Mintware Swap	? Live
Referral system	? Live
Arbitrum campaigns	? Live
AI Agent Campaigns	? Live ? Base mainnet
Social Liquidity Vaults	? Coming soon ? Phase 2
Self-serve campaign creation	? Coming soon

```
=====
FILE: docs/overview/what-is-mintware.md
=====
# What is Mintware?
```

Mintware solves a fundamental problem in DeFi incentives: protocols can't tell who their best users actually are.

Today, most protocols distribute rewards equally ? or proportional to capital. A whale who dumped last cycle gets the same incentives as a genuine long-term user. Sybil farmers get the same as real participants.

Mintware fixes this by introducing **reputation as a first-class primitive**.

The Core Idea

Your wallet has a history. That history tells a story ? when you started, how consistently you've been active, what you've done across chains, and how you've behaved during volatile periods.

Attribution reads that story and turns it into a score. Mintware uses that score to determine your share of protocol incentives.

How It Works

Your wallet history
?
Attribution scoring (100+ chains)
?
Score + tier assigned
?
Join campaigns that match your profile
?
Earn rewards weighted by your score
'''

Who Is It For?

****DeFi users**** who have been active for years but haven't been rewarded proportionally to their contribution ? Attribution gives your history real economic value.

****Protocols**** who want to direct incentives toward genuine users rather than paying equally for sybil wallets and real participants.

=====

FILE: docs/overview/attribution.md

=====

Attribution ? On-Chain Reputation

Attribution is Mintware's scoring engine. It analyses your complete on-chain history across 100+ chains and produces a single composite reputation score that reflects the quality, depth, and consistency of your wallet activity.

Your score is not based on wealth. A wallet with decades of consistent, diverse on-chain activity scores higher than a whale who deployed capital yesterday.

What Gets Measured

Attribution breaks your wallet history into six independent signal categories:

Signal	Max Score	What It Measures
Volume	100	The scale and consistency of your transaction activity over time
Trading	75	Your trading behaviour ? frequency, diversity, protocol breadth, and timing across market conditions
Holding	100	Long-term asset conviction ? how long you hold relative to your entry, and whether you accumulate or churn
Liquidity	150	Participation in liquidity provision ? depth, duration, and quality of your LP positions
Governance	100	On-chain governance participation across protocols ? proposals, votes, and delegation history
Sharing	400	The depth and quality of your referral network ? wallets you've brought on-chain and their downstream activity

Each signal is scored independently. They combine into a total Attribution score.

****Maximum possible score: 925.****

How Signals Are Scored

Each signal uses its own set of behavioural indicators drawn from your raw on-chain history. The scoring is designed to reward genuine, sustained participation ? not gaming:

- **Recency matters, but history matters more.** Consistent activity over years scores better than a sudden burst.
- **Breadth matters.** Being active across multiple chains and protocols reflects genuine engagement, not narrow optimisation.
- **Quality beats quantity.** Thousands of low-value transactions score lower than sustained, meaningful activity.
- **Sharing is weighted heavily** because building a real referral network that generates genuine on-chain activity is hard to fake.

The specific weights and formulas are proprietary. What you need to know: the score rewards being a genuine, long-term DeFi participant.

Score Tiers

Tier	Description
---	---
Bronze	Emerging on-chain presence ? building history and activity across chains
Silver	Established participant with consistent history and protocol diversity
Gold	Seasoned, long-term wallet with depth across signals

Tier thresholds are applied relative to the distribution of all scored wallets ? they shift as the community grows.

Wallet Character

Beyond your score, Attribution assigns a **character** to your wallet based on behavioural patterns ? how you respond to volatility, your consistency across bull and bear markets, and your long-term orientation.

Character labels reflect observed behavioural traits, not judgements. Examples:

- **Ghost** ? active in calm markets, dormant in chaos
- **Builder** ? consistent protocol engagement regardless of market conditions
- **Liquidity Provider** ? concentrated LP activity and depth

Character is shown on your profile page and used to surface relevant campaign and protocol opportunities.

Multi-Chain Coverage

Attribution analyses activity across 100+ EVM and non-EVM chains. You don't need to migrate assets or bridge anything ? Attribution reads your existing history wherever it lives.

Your score is chain-agnostic. Deep history on Arbitrum, Optimism, Polygon, Solana, or any supported chain contributes equally to your score. History is history.

Score Updates

Your Attribution score updates as your on-chain activity grows. Major new activity ? sustained LP positions, governance participation, referral network expansion ? can meaningfully shift your score over time.

Short-term gaming (sudden bursts of low-quality activity) is detected and does not produce score increases proportional to the raw activity.

Verifiable On-Chain

Your Attribution score can be published as a cryptographically signed EAS (Ethereum Attestation Service) attestation. This makes your score:

- **Verifiable** by any third party without trusting Mintware
- **Portable** to any protocol that integrates EAS
- **Timestamped** ? the attestation records when the score was measured

See [EAS Attestations](../developers/eas-attestations.md) for details.

How to Improve Your Score

There is no shortcut. The signals that matter most ? long-term holding, liquidity depth, governance participation, and organic referral networks ? take time to build.

Practical steps:

- Participate in governance across the protocols you already use
- Add and maintain LP positions rather than one-time deposits
- Build your referral network authentically ? refer wallets that will actually be active
- Stay consistent across multiple chains rather than concentrating on one

See [Your Score](../getting-started/your-score.md) for how to read and interpret your score in detail.

=====

FILE: docs/overview/mintware-platform.md

=====

Mintware Platform ? Earn & Campaigns

The Mintware platform is where your Attribution score translates into earnings.

Protocols and teams create campaigns ? pools of tokens or points ? and Mintware distributes rewards to participants based on their activity and Attribution score.

Campaign Types

Token Reward Pools

Anyone can create a token reward pool. Rewards are distributed per qualifying swap transaction ? no epoch, no waiting. Every time you complete a qualifying swap, a reward is locked for your wallet and becomes claimable after a short window.

****Best for:**** Protocols wanting to incentivise immediate trading activity.

Points Campaigns

Points campaigns are created by whitelisted teams. Rewards are distributed per epoch (a fixed time window) based on the actions you complete and your Attribution score multiplier.

****Best for:**** Protocols running structured loyalty or growth programs.

How Rewards Are Weighted

In Points Campaigns, your Attribution score determines a multiplier applied to your raw point total. Wallets with higher scores earn a larger share of the epoch pool for the same actions.

This ensures that protocols reward genuine, established users ? not sybil wallets or one-day-old accounts.

Epoch Distribution

Points Campaign rewards are settled at the end of each epoch:

1. Your points are tallied for the epoch
2. Your score multiplier is applied
3. Your share of the epoch pool is calculated relative to all participants
4. A Merkle distribution is published and signed by Mintware's oracle
5. You can claim your allocation directly on-chain

The Dashboard

The Mintware dashboard ('/dashboard') shows:

- All live, upcoming, and ended campaigns
- Your joined campaigns and pending rewards
- Campaign details ? reward pool size, epoch length, qualifying actions

Access Requirements

Some Points Campaigns have a minimum Attribution score requirement ('min_score'). If your score is below the threshold, you won't be able to join until your score improves.

=====

FILE: docs/overview/score-tiers.md

Score Tiers

Your Attribution score determines your tier. Tiers affect your eligibility for certain campaigns and the multiplier applied to your Points Campaign earnings.

Tiers

Tier	Score Range	Notes
---	---	---
Bronze	0 ? 299	Starting tier for most wallets
Silver	300 ? 599	Established on-chain presence
Gold	600 ? 925	Long-term, high-quality participants

> Score ranges are indicative. The exact boundaries are set by the Attribution engine and may be adjusted as the scoring model evolves.

Percentile

Your profile also shows your **percentile** ? where your score sits relative to all scored wallets. This gives you a sense of your standing beyond the tier label.

How to Improve Your Score

Your score reflects the breadth and quality of your on-chain history. The following types of activity are measured:

- **Consistent activity** over time ? not just recent bursts
- **Cross-chain presence** ? activity across multiple networks
- **Trading and holding behaviour** ? long-term conviction is rewarded
- **Liquidity provision** ? participating in protocol liquidity
- **Governance participation** ? voting and on-chain governance
- **Referrals** ? bringing quality wallets into your network

There is no shortcut. Attribution is designed to be resistant to manufactured activity.

Score Opportunities

Your profile's **Score tab** surfaces personalised protocol opportunities based on your current signal gaps ? protocols where your existing profile is a strong fit, with estimated score ranges you could achieve by participating.

FILE: docs/getting-started/connect-wallet.md

Connect Your Wallet

Mintware supports any EVM wallet via WalletConnect and browser extensions.

Supported Wallets

- MetaMask
- Coinbase Wallet
- Rainbow
- Trust Wallet
- Any WalletConnect-compatible wallet

How to Connect

1. Go to mintware.finance
2. Click ****Connect Wallet**** in the top navigation
3. Select your wallet and approve the connection
4. Your Attribution score will load automatically

> ****No signature required**** to view your score. Signatures are only requested when joining a campaign or claiming rewards.

Before You Confirm in Your Wallet

Mintware is designed to explain important on-chain actions before your wallet opens.

Depending on the action, you may see:

- what you are sending
- what you are expected to receive
- which network the action will happen on
- the estimated network fee
- whether you are giving token permission or completing the final action

This helps reduce failed transactions, approval confusion, and wrong-network mistakes.

Supported Networks

Mintware campaigns run across multiple EVM chains. Each campaign specifies which chain it operates on ? check the campaign detail page to see which network you need to be connected to.

Your Attribution score is calculated from your history across 100+ chains ? you don't need to move assets to any specific chain to have a score. Attribution reads your existing history wherever it is.

Chain	Campaigns	Swap
---	---	---
Base	? Live	? Live
Arbitrum	? Coming soon	? Live
Core	? Live	? Coming soon

First Connection

When you connect for the first time:

- A referral code is generated for your wallet
- If you arrived via a referral link, the referral is recorded
- A brief welcome sheet will appear with your referral link

Disconnecting

Hover over the wallet pill in the navigation bar and click the disconnect button that appears. You'll be returned to the landing page.

```
=====
FILE: docs/getting-started/reviewing-transactions.md
=====
```

Reviewing Transactions

Mintware is designed to explain important on-chain actions before your wallet opens ? not inside the wallet popup where the description is often truncated or technical.

Swap ? Review Panel

When you click **Review Swap**, a confirmation panel appears before your wallet opens. It shows:

- the token you are sending and the exact amount, with its USD value
- the estimated token and amount you will receive
- which network the swap will happen on
- the estimated network fee ? shown as **'~\$0.05'** (fiat) with the native token amount as secondary context ('~0.00003 ETH')
- the Mintware platform fee as a percentage with an estimated dollar amount
- price impact ? highlighted in red if it exceeds 2%
- a plain-language explanation of what will happen next

After reviewing, click **Open wallet to confirm** to open your wallet. You can cancel at any point before that.

> **Low gas warning:** If your wallet balance does not appear to cover the estimated fee, a warning appears before you reach the confirmation panel.

Approval vs Final Action

Some actions involve two distinct steps:

1. **Giving permission** ? a contract is authorised to use a specific token and amount on your behalf. No funds move at this step.
2. **Final action** ? the actual transfer, deposit, or execution happens.

Mintware distinguishes these steps clearly. Language like "Give permission to use tokens" means step 1. The follow-up confirmation ? funding, depositing, or claiming ? means step 2.

****Allowance pre-check:**** Mintware checks whether the contract already has sufficient allowance before asking you to approve. If your wallet has already approved enough, the permission step is skipped automatically. You only see an approval prompt when one is actually needed.

Campaign Funding

When funding a campaign as a sponsor, Mintware shows:

- whether an approval is needed or will be skipped
- a clear label: "Give permission to use tokens" (not "Approve token spend")
- a "What will happen" summary showing 1 step or 2 steps depending on your current allowance
- per-step context while the transaction is in-flight

Claiming Rewards

Before a claim transaction is submitted:

- a context line shows what will arrive and where: '0.25 USDC on Base ? your wallet'
- the button reflects the actual state: 'Getting proof?' ? 'Check your wallet ?' ? 'Waiting for confirmation?'

If you are on the wrong network for the claim, the app will prompt you to switch before the wallet opens.

Vault Actions

Vault deposit and approval flows separate permission and deposit steps explicitly. The network the vault operates on is shown before any confirmation. A zero-first approval path is used for USDT-style tokens that reject a non-zero approval while one is already active.

Why This Matters

Wallet prompts contain limited context. Mintware adds product-level explanations before the popup so you can:

- confirm you are interacting with the right network
- understand whether you are giving permission or transferring value
- see the full cost of the action (fees in fiat + native token)
- make the call quickly and confidently

The goal is that nothing in the wallet popup should surprise you.

```
=====
FILE: docs/getting-started/your-score.md
=====
```

Your Attribution Score

Your Attribution score is calculated automatically from your on-chain history across 100+ chains. No action is required ? just connect your wallet and your score loads.

Where to Find It

Your score is displayed on the **Profile** page ('/profile'), accessible from the navigation bar after connecting.

What You'll See

Element	Description
Score & tier	Your total score and tier ? Bronze, Silver, or Gold
Percentile	Where you rank against all scored wallets globally
Signal breakdown	Your score across each of the six signal categories, with insights
Wallet character	A character assigned based on your behavioural patterns
Opportunities	Protocols and actions that could improve your specific signal gaps
Timeline	Your score history over time

The Six Signals

Signal	Max	What It Measures
Volume	100	Scale and consistency of transaction activity
Trading	75	Frequency, diversity, and timing of trades
Holding	100	Long-term asset conviction vs short-term flipping
Liquidity	150	Participation in liquidity provision
Governance	100	On-chain governance across protocols
Sharing	400	Your referral network size and quality

Max total score: 925

Scoring Lag

Attribution analyses your on-chain history as of the last indexing run. Very recent transactions may not be reflected immediately. Scores are refreshed periodically.

Multiple Wallets

Each wallet address has its own independent Attribution score. Scores are not combinable across wallets.

EAS Attestation

Your score can be published as a verifiable offchain EAS (Ethereum Attestation Service) attestation. This creates a cryptographically signed, portable record of your Attribution score that any protocol can verify independently ? without trusting Mintware.

To publish your attestation, visit the ****Score tab**** on your profile page and click ****Attest Score****.

See [EAS Attestations](../developers/eas-attestations.md) for technical details.

=====

FILE: docs/getting-started/your-profile.md

Your Profile

The Profile page is your personal hub on Mintware. It shows your Attribution score, wallet history, and referral status.

Profile Tabs

Portfolio

A summary of your on-chain asset positions and protocol deployments ? where your capital is currently sitting and how it's allocated.

Score

Your full Attribution breakdown:

- Total score and tier
- Signal-by-signal breakdown with insights
- Score improvement opportunities
- EAS attestation (publish your score on-chain)

Badge

Your wallet character and tier badge. Shareable proof of your Attribution standing.

Invite

Your referral code and link. Share it to grow your Sharing signal score and earn referral rewards when people you invite participate in campaigns.

Wallet Display

Where possible, Mintware resolves your wallet address to a ****Basename**** (Base Name Service handle). If you have a Basename registered on Base, it will display throughout the app instead of your raw address.

Mintware also surfaces chain context more explicitly around actions that depend on a specific network, so it is easier to tell when you are connected to the wrong chain before

confirming.

Score Attestation

From the Score tab, you can publish your Attribution score as an EAS attestation. Once attested:

- Your score is permanently recorded on the EAS network
- Third parties can verify your score without relying on Mintware's API
- The attestation is linked to your wallet address and signed by Mintware's oracle

```
=====
FILE: docs/getting-started/your-identity.md
=====
```

Your Identity

Mintware resolves human-readable names for wallet addresses wherever they appear ? in the nav, on the leaderboard, and on your profile.

Basenames

[Basenames](https://base.org/names) are on-chain names registered on Base (e.g. 'alice.base.eth'). If your connected wallet has a Basename registered, Mintware displays it in place of your truncated address.

Basenames are resolved silently in the background ? no action is needed on your part.

ENS

Standard Ethereum Name Service names (e.g. 'alice.eth') are also resolved and displayed where a Basename is not present.

Resolution Priority

1. ****Basename**** ? checked first (Base-native)
2. ****ENS**** ? checked if no Basename is found
3. ****Truncated address**** ? shown if no name resolves (e.g. '0x1234?abcd')

Where Names Appear

- ****Nav wallet pill**** ? shows your name once connected
- ****Leaderboard**** ? participant names displayed in rankings
- ****Profile page**** ? shown at the top of your profile card
- ****Referral links**** ? your ref link uses a code derived from your address, not your name

Referral Codes

Your referral code is generated deterministically from your wallet address ? it does not depend on your Basename or ENS. Even if you change your on-chain name, your referral code stays the same.

```
=====
FILE: docs/campaigns/overview.md
=====
```

Campaign Overview

Campaigns are the core reward mechanism on Mintware. Protocols and teams create campaigns to incentivise specific on-chain behaviour ? trading, bridging, providing liquidity, referring others, or contributing as an AI agent.

Your Attribution score determines how much you earn relative to other participants. The same action is worth more to a higher-scored wallet.

Three Campaign Types

Mintware supports three distinct campaign models, each designed for a different incentive structure.

1 ? Token Reward Pools

****Best for:**** protocols that want to reward individual swaps immediately, with no epoch delay.

A token budget is deposited upfront. Every qualifying swap locks a reward directly for the swapper's wallet in real time. The reward becomes claimable after a short verification window ? no epoch, no waiting for batch settlement.

? [Full details: Token Reward Pools](token-reward-pools.md)

2 ? Points Campaigns

****Best for:**** protocols running sustained multi-week incentive programmes where quality of participation matters.

Participants earn points by completing qualifying actions over a fixed time window called an epoch. At epoch end, the pool is split proportionally across all participants ? but each wallet's share is amplified by their Attribution score multiplier. Higher-tier wallets earn a larger slice for the same actions.

? [Full details: Points Campaigns](points-campaigns.md)

3 ? AI Agent Campaigns

****Best for:**** protocols that want to incentivise autonomous AI agents for on-chain volume and activity.

AI agents register on-chain and contribute volume to active campaigns. The oracle watcher automatically detects agent activity and pre-signs reward actions ? agents claim them by submitting oracle signatures to the contract. The oracle pays nothing; agents pay their own gas.

? [Full details: AI Agent Campaigns](ai-campaigns.md)

Side-by-Side Comparison

		Token Reward Pool		Points Campaign		AI Agent Campaign	
	---	---		---		---	
	Created by			Anyone (self-serve)		Whitelisted teams	
	Reward trigger			Per qualifying swap		Per epoch distribution	
contribution							
	Score multiplier			No		Yes ? up to 1.95×	
	Access			Open		May require minimum score	
	Reward timing			Claimable after lock window		Claimable after epoch settles	
Claimable after oracle signs							
	Participants			Human wallets		Human wallets	
AI agent wallets							

Finding Campaigns

The **Dashboard** (`/dashboard`) lists all active campaigns. Filter by status (Live / Upcoming / Ended) and chain (Base, Arbitrum, Core, and others).

Each campaign card shows the sponsor, reward pool size and token, qualifying actions, status, and time remaining.

Supported Chains

	Chain		Status	
	---		---	
	Base		? Live	
	Arbitrum One		? Live	
	Core DAO		? Live	
	BNB Chain		? Planned	

Leaderboard

The **Leaderboard** (`/leaderboard`) shows rankings across all campaigns. Select a campaign to see where you stand relative to other participants.

Clearer Funding and Claiming

Mintware is improving campaign transactions so they are easier to understand before your wallet opens.

That includes:

- clearer explanations of when you are giving token permission versus making the final deposit
- fewer unnecessary approval prompts when an allowance already exists
- better chain and wallet-state guidance during claims

- stronger checks before submitting important on-chain actions

```
=====
FILE: docs/campaigns/token-reward-pools.md
=====
```

Token Reward Pools

Token Reward Pools are the simplest campaign type ? a protocol deposits tokens, you complete qualifying swaps, and rewards are locked per transaction in real time. No epochs, no waiting for batch settlement.

How It Works

```
'''
Protocol deposits token budget into the campaign pool
    ?
You complete a qualifying swap via Mintware Swap
    ?
A reward is automatically locked for your wallet
    ?
Short lock period (for on-chain verification)
    ?
Reward becomes claimable ? claim directly on-chain
    ?
Tokens transfer to your wallet
'''
```

The entire flow is automatic. You swap, the reward is locked, you claim when ready.

Reward Structure

Each qualifying swap generates up to three reward components:

Component	Who Receives It	Description
***Buyer Reward**	The swapper	The primary reward for completing the qualifying swap
***Referrer Reward**	The wallet that referred you (if applicable)	A share of the swap value directed to your referrer
***Platform Fee**	Mintware treasury	2% of the swap value ? Mintware's fee for operating the campaign

The buyer reward percentage is set by the campaign creator when the pool is created. Referrer rewards incentivise growth through your referral network ? see [Referral System](../referrals/overview.md).

Qualifying Swaps

Not all swaps qualify. Each campaign defines:

- **Minimum swap size** ? a floor transaction value in USD

- ****Eligible tokens**** ? specific trading pairs or tokens that count
- ****Chain**** ? swaps must occur on the campaign's designated chain

Always check the campaign detail page to confirm what qualifies before swapping.

The Verification Window

There is a brief lock period between your swap and when the reward becomes claimable. During this window, Mintware verifies the on-chain transaction to confirm it meets campaign requirements.

Verification checks include:

- The transaction was successfully mined on-chain
- The swap was routed through a supported DEX aggregator
- The Mintware treasury fee was included in the transaction

This lock window exists to prevent fraudulent reward claims. Once verification passes, your reward status changes to ****Claimable****.

Claiming

When a reward is claimable, it appears in the ****Rewards**** section of the campaign detail page.

1. Click ****Claim****
2. Confirm the on-chain transaction in your wallet
3. Tokens transfer directly to your connected wallet

> Gas is required to claim. Ensure your wallet has sufficient native token on the campaign's chain (e.g. ETH on Base, ETH on Arbitrum).

Pool Depletion

The reward pool depletes as rewards are claimed. Once the pool is empty, the campaign ends automatically ? no further rewards lock even if swaps continue. Campaign creators can top up the pool to extend the campaign.

The current pool balance is always shown on the campaign detail page.

Batch Claiming

If you have multiple claimable rewards across epochs on the same contract, the UI displays a ****Claim All (N)**** button that submits a single batch transaction ? saving gas compared to claiming individually.

Creating a Pool

Token Reward Pool creation will be available to any wallet via the self-serve campaign interface (coming soon). Creating a pool requires:

1. Choosing the reward token and total budget
2. Setting the reward percentage per swap
3. Defining qualifying actions (minimum swap size, eligible tokens, chain)
4. Depositing the full token budget upfront

The first depositor becomes the campaign creator and is the only wallet that can recover remaining funds after the campaign closes.

```
=====
FILE: docs/campaigns/points-campaigns.md
=====
```

Points Campaigns

Points Campaigns run over fixed time windows called epochs. Participants earn points by completing qualifying actions, and at epoch end, the reward pool is distributed based on each wallet's point share ? amplified by their Attribution score multiplier.

The result: two participants who complete the exact same actions can earn different rewards depending on how strong their on-chain reputation is.

How It Works

'''

Join the campaign (subject to minimum score requirement, if set)

?

Complete qualifying actions during the epoch to earn points

?

Points are credited in real time as actions are verified

?

Epoch ends ? total points across all participants are tallied

?

Your Attribution score multiplier is applied to your point total

?

Your share of the epoch pool = your weighted points / all weighted points

?

Mintware publishes a Merkle distribution, signed by the oracle

?

Claim your allocation on-chain

'''

Epochs

An epoch is a fixed time window ? typically one week. Points reset at the start of each epoch. Past performance doesn't carry over.

Each epoch is a fresh competition. If a campaign runs for multiple epochs, each one is settled and distributed independently.

Qualifying Actions

Each campaign defines its own set of qualifying actions and point values. Common examples:

Action	Base Points	Frequency
Bridge	15 pts	Once per wallet per campaign
Trade	8 pts	Once per calendar day
Referral bridge	60 pts	Per referred wallet that bridges
Referral trade	8 pts	Per referred wallet per trading day

Always check the campaign detail page for the exact actions and values ? they vary by campaign.

Score Multipliers

Your Attribution score applies a multiplier to your raw point total at the moment each action is credited. There are two independent multipliers:

****Attribution multiplier**** ? based on your Attribution score percentile relative to all scored wallets:

Attribution Percentile	Multiplier
0-33% (lower third)	1.0x
34-66% (middle third)	1.25x
67-100% (top third)	1.5x

****Sharing multiplier**** ? based on your Sharing score percentile (the strength of your referral network):

Sharing Percentile	Multiplier
0-33%	1.0x
34-66%	1.15x
67-100%	1.3x

Both multipliers combine ****multiplicatively****. Maximum combined multiplier: ****1.95x****.

> ****Example:**** A wallet in the top third for both Attribution and Sharing earns $1.5 \times 1.3 = 1.95x$ on every action. A trade worth 8 base points becomes 15.6 weighted points.

Multipliers are locked at the time each action is credited ? they do not retroactively change if your score shifts during the campaign.

The Epoch Distribution

When an epoch closes, the distribution is calculated off-chain:

1. All participant point totals are read (with multipliers already applied)
2. Each wallet's share is calculated as their weighted points divided by the sum of all weighted points

3. Each wallet's share is multiplied by the epoch pool size
4. A Merkle tree is built from the final allocations
5. Mintware's oracle cryptographically signs the Merkle root
6. The signed distribution is published on-chain and becomes claimable

No gas is spent by Mintware to publish the distribution ? the oracle signature is enough. Claimers pay their own gas when they submit their Merkle proof.

Minimum Score Access

Some Points Campaigns have a 'min_score' threshold. Wallets below the threshold cannot join. If you're blocked, your Attribution score needs to improve first ? see [Your Score](../getting-started/your-score.md).

Who Creates Points Campaigns

Points Campaigns are created by whitelisted teams only. This is intentional ? the score-multiplier system has real economic impact, and campaigns must be structured responsibly. Self-serve creation for Token Reward Pools is separate and open to anyone.

Claim Deadline

Every epoch distribution includes a deadline. Claims must be submitted on-chain before this deadline. Deadlines are set generously, but don't leave them indefinitely ? once expired, the allocation cannot be recovered.

```
=====
FILE: docs/campaigns/ai-campaigns.md
=====
```

AI Agent Campaigns

AI Agent Campaigns are designed for autonomous AI agents operating on-chain. They allow AI wallets to contribute volume to active campaigns, build a verifiable on-chain reputation, and earn attribution for their activity ? all without requiring manual intervention or oracle gas costs.

How It Works

'''

AI agent registers on-chain (one time)

?

Oracle watcher monitors Base mainnet every 60 seconds

?

Agent's on-chain activity is detected (transfers, swaps, campaign interactions)

?

Oracle validates the activity and cryptographically signs a reward action

?

Signature is stored ? agent picks it up at any time

?

Agent submits the oracle signature to the contract

?

Contract verifies the signature and updates the agent's Attribution score

'''

The oracle never pays gas. The agent pays for its own on-chain submissions. This keeps the system economically sustainable at any scale.

What Differentiates AI Agent Campaigns

Unlike human-facing campaigns, AI Agent Campaigns are built around **agent identity and verifiability**:

- **On-chain proof** ? every scoring action is verified and recorded on-chain, not just stored in a database
- **Replay protection** ? each action uses a cryptographic nonce, making double-submission impossible
- **Gasless oracle** ? the oracle signs off-chain using EIP-712 typed data; agents submit and pay gas themselves
- **Automatic detection** ? agents don't need to manually call the oracle. The watcher detects activity and pre-signs actions automatically
- **ERC-8004 native** ? agents can link their wallet to an ERC-8004 AI Identity token, connecting their reputation to a broader identity standard

Agent Attribution Score

Every registered AI agent builds a score across four dimensions:

Dimension	What It Reflects
Behavior	Consistency, frequency, and quality of on-chain activity over time
Contribution	Cumulative volume contributed to active campaigns
Interpretability	Transparency ? submissions of MWP (Model Weight Provenance) hashes proving the agent's reasoning is auditable
Risk	Deductions for flagged or anomalous behaviour

Score formula: 'total = max(0, behavior + contribution + interpretability ? risk)'

The score is stored on-chain and readable by any protocol or smart contract.

The Transparent Agent Badge

Agents that publish their model reasoning as MWP folder hashes earn the **Transparent Agent** badge. This signals that the agent's decision-making process is not a black box ? it can be audited by anyone holding the corresponding IPFS content.

Each unique MWP hash submitted earns interpretability points (capped at a maximum). The badge is reflected both in the on-chain score and in the agent's profile on the Mintware leaderboard.

Creating an AI Campaign

Any protocol can create an AI Agent Volume Campaign on-chain:

```
'''typescript
import { createCampaign } from '@mintware/ai-attribution-sdk'

const txHash = await createCampaign({
  name: 'Protocol Volume Sprint',
  targetVolume: 1_000_000n * 10n**18n, // target in wei
  durationSecs: 604800n, // 7 days
  privateKey: '0x...',
})
'''
```

Campaigns are permissionless ? no whitelisting required. Any wallet can define a target volume, duration, and name.

Agent Volume Tracking

Each campaign tracks volume per agent independently. Volume is recorded in wei and tied to the campaign ID. Multiple campaigns can run simultaneously, and agents accumulate volume across all active campaigns.

Campaign volume is readable on-chain and queryable via the Mintware API and subgraph.

Getting Started as an Agent

See [AI Attribution ? Getting Started](../ai-attribution/getting-started.md) for the full setup guide, including registration, SDK integration, and how to claim pending oracle actions.

Oracle Watcher

The oracle watcher is a background service that automatically monitors the chain for registered agent activity. Agents do not need to poll the oracle or call any API proactively ? the watcher handles detection and pre-signing.

See [Oracle Watcher](../ai-attribution/oracle-watcher.md) for how it works.

Network

AI Agent Campaigns are live on ****Base mainnet****.

```
| | |
|---|---|
| Chain | Base (8453) |
```

```
| Contract | '0x11Ef2c7D84b755f02f3652ca8b16e6E81A96C421' (AIAttribution v3) |
| Subgraph | 'https://api.studio.thegraph.com/query/1745134/mintware-ai-
attribution/v0.3.0' |
```

```
=====
FILE: docs/campaigns/score-multipliers.md
=====
```

Score Multipliers

Points Campaigns apply multipliers to your raw action points based on your on-chain reputation. This means wallets with stronger histories earn more points per action ? and a proportionally larger share of each epoch's reward pool.

Two Multipliers

```
| Multiplier | Based on | Max |
|---|---|---|
| Attribution | Your Attribution score percentile | 1.5× |
| Sharing | Your referral network (Sharing score percentile) | 1.3× |
```

Both multipliers are applied ****multiplicatively****. The maximum combined multiplier is ****1.95×****.

Percentile Bands

```
| Percentile | Attribution multiplier | Sharing multiplier |
|---|---|---|
| 0?33% | 1.0× | 1.0× |
| 34?66% | 1.25× | 1.15× |
| 67?100% | 1.5× | 1.3× |
```

Your percentile is calculated relative to all scored wallets at the time you join the campaign.

When Multipliers Apply

Multipliers are applied ****at point-credit time**** ? when your action (swap, bridge, referral) is recorded. They are not retroactively recalculated if your score changes during the campaign.

This means:

- Building your Attribution score before joining a campaign gives you a better multiplier from day one
- Referring wallets before joining increases your Sharing score, which boosts your Sharing multiplier

Example

A wallet in the 70th percentile for both Attribution and Sharing earns:

```
'''
1.5 × 1.3 = 1.95× per action
'''
```

A trade action worth 8 base points becomes **15.6 points** after multipliers.

Which Campaigns Use Multipliers

Only **Points Campaigns** use score multipliers. Token Reward Pool campaigns pay a flat percentage of swap value ? no multipliers apply.

Check the campaign detail page to see whether a campaign is a Points Campaign and what your current multiplier is.

```
=====
FILE: docs/campaigns/joining.md
=====
```

Joining a Campaign

Steps to Join

1. Go to the **Dashboard** ('/dashboard')
2. Find a campaign you want to join ? filter by status or chain if needed
3. Click the campaign to open its detail page
4. Review the qualifying actions and reward structure
5. Click **Join Campaign**
6. Confirm the wallet signature

That's it. You're now a participant. Start completing qualifying actions to earn points or trigger swap rewards.

Eligibility

Token Reward Pools ? open to any connected wallet.

Points Campaigns ? may require a minimum Attribution score. If your score is below the threshold, the Join button will be disabled and your current score will be shown alongside the requirement.

After Joining

Once joined, the campaign appears in your dashboard with your current participation status. You'll see:

- Points earned this epoch (Points Campaigns)
- Pending rewards (Token Reward Pools)
- Your rank on the campaign leaderboard

Multiple Campaigns

You can join as many campaigns as you like. There's no limit.

```
=====
FILE: docs/campaigns/claiming-rewards.md
=====
```

Claiming Rewards

Token Reward Pool Claims

Rewards from Token Reward Pools become claimable after a short lock period following your qualifying swap.

1. Go to the campaign detail page
2. Any claimable rewards will appear in the ****Claim**** section
3. A context line shows exactly what will arrive and where ? e.g. '0.25 USDC on Base ? your wallet'
4. Click ****Check your wallet ?**** and confirm the on-chain transaction
5. Tokens transfer directly to your connected wallet

> ****Gas reminder:**** Claiming requires an on-chain transaction. Make sure your wallet has a small amount of the campaign chain's native token for gas (e.g. ETH on Base or Arbitrum).

The button reflects the actual state as the claim progresses: 'Getting proof?' ? 'Check your wallet ?' ? 'Waiting for confirmation?'. If you are on the wrong network, the app will prompt you to switch before the wallet opens.

Points Campaign Claims

At the end of each epoch, Mintware calculates your allocation and publishes a Merkle distribution.

1. Once the epoch ends and the distribution is published, a ****Claim**** button appears on the campaign detail page
2. Click ****Claim**** ? your Merkle proof is fetched automatically
3. Confirm the on-chain transaction
4. Tokens transfer to your wallet

Claim Deadline

Oracle-signed distributions have an expiry. Claims must be submitted before the deadline embedded in the signature. Deadlines are set generously (30 days by default) but don't leave it indefinitely.

> ****Don't delay.**** If the deadline passes, that epoch's reward cannot be claimed. The contract enforces this ? there is no override.

Already Claimed

If you've already claimed a distribution, the campaign page will show **Claimed** with the timestamp. You cannot claim the same distribution twice ? the contract enforces this on-chain.

Batch Claiming

If you have claimable rewards across multiple epochs on the same campaign contract, the UI will show a **Claim All (N) ? check your wallet ?** button. This submits a single 'batchClaim()' transaction covering all eligible distributions ? saving gas compared to claiming each epoch individually.

Batch claims fetch a Merkle proof for your wallet address for each eligible distribution before opening the wallet. The network context is checked first so the app can prompt you to switch to the correct chain before any transaction is submitted.

Troubleshooting

Claim button not appearing ? the epoch may not have settled yet, or the distribution hasn't been published. Check back shortly after epoch end.

Transaction reverts ? ensure your wallet is connected to the correct network for this campaign and has sufficient gas. If the issue persists, the claim window may have expired.

Wrong wallet connected ? rewards are claimable only by the wallet that earned them. Make sure you're connected with the correct wallet.

=====

FILE: docs/vaults/overview.md

=====

Social Liquidity Vaults

> **Status: Coming Soon ? Phase 2**

>

> Social Vaults are built and in final testing on Base Sepolia. The nav tab is visible but locked. Public launch date TBC.

What Are Social Vaults?

Social Liquidity Vaults are Uniswap V4 LP positions managed collectively through Mintware ? but with one fundamental difference from standard LP: **your share of the fee pool is weighted by your Attribution score and referral network.**

Two wallets depositing the exact same amount earn different yields. The wallet with the stronger on-chain reputation earns more.

This creates a new kind of LP primitive: one where your DeFi history has direct economic value beyond your capital.

How It Works

'''

You deposit assets into a Social Vault

?

The vault deploys your liquidity into a Uniswap V4 pool

?

A Mintware hook captures dynamic fees + MEV on price deviation

?

Fees accumulate in the FeeVault throughout the epoch

?

Weekly epoch close ? FeeVault distributes fees to depositors

?

Your share = deposit × lock tier multiplier × Attribution multiplier × referral multiplier

?

Claim your allocation on-chain

'''

Three Multipliers on Your Yield

1 ? Lock Tier Multiplier

Committing your liquidity for longer earns a higher base multiplier. Early exits are penalised ? the penalty is distributed back to remaining LPs.

Lock Tier	Duration	Fee Multiplier
Flexible	No lock	1.0×
Committed	30 days	1.25×
Loyal	90 days	1.5×
Dedicated	180 days	2.0×

2 ? Attribution Multiplier

Your Mintware Attribution score percentile determines an additional multiplier on top of your lock tier:

Attribution Percentile	Additional Multiplier
0-33%	1.0×
34-66%	1.25×
67-100%	1.5×

3 ? Referral Multiplier

Your Sharing score ? the depth and activity of your referral network ? applies a further multiplier on your yield:

Sharing Percentile	Additional Multiplier
0-33%	1.0×
34-66%	1.15×
67-100%	1.3×

All three multipliers apply together. A dedicated depositor in the top Attribution and Sharing percentiles earns the maximum combined yield on their position.

The Fee Engine

Fees are not just collected ? they're actively captured. The Mintware V4 hook applies a dynamic fee that scales with price deviation from the previous block. When price moves significantly, a larger fee is captured. This is designed to capture value that would otherwise go to MEV bots exploiting LP positions.

The captured fees flow into the FeeVault, which holds them until epoch close and then distributes to depositors according to their weighted shares.

Weekly Epoch Distribution

At the end of each weekly epoch:

1. The vault's total accumulated fees are tallied
2. Each depositor's weighted share is calculated ($\text{deposit} \times \text{lock tier} \times \text{Attribution} \times \text{referral}$)
3. A Merkle distribution is built and signed by the Mintware oracle
4. Depositors can claim their allocation on-chain

The oracle signature model means Mintware never pays gas to publish distributions. Depositors pay their own claim gas.

Rebalancing

Vault liquidity is concentrated within a tick range. When the price moves outside that range, the vault needs to rebalance. Mintware uses an AI Range Optimizer to propose new tick ranges based on on-chain price volatility data.

Rebalancing is ****permissionless**** ? anyone can submit a valid oracle-signed range proposal to the vault contract. The contract verifies the oracle signature before executing. This avoids centralisation: Mintware can propose ranges, but so can any party with a valid signed proposal.

Withdrawal

Withdrawals are subject to the lock tier you selected at deposit. Flexible depositors can withdraw at any time. Locked depositors who exit early pay a penalty ? the penalty amount is immediately redistributed to remaining LPs in the same vault.

After a vault is closed, a cooldown period allows final claims before the creator can recover remaining funds.

Smart Contracts (Base Sepolia ? Testnet)

Contract	Address	Role
SocialVault	'0xb9FB965Caa7197932b52631e0121Ea54586e2B88'	Manages LP deposits, liquidity position, withdrawals
FeeVault	'0x4Deb74E9D50Ebbf9bD883E0A2dcD0a1b4b9Db9BE'	Accumulates fees + MEV capture; distributes at epoch close
MWSocialHook	'0x8e7e05f5b6ed07acAa7Ac41D74a0d86a50AA8aC4'	Uniswap V4 hook ? dynamic fees, MEV capture on price deviation

Mainnet deployment is a separate step ahead of public launch.

> **Note:** No Mintware protocol fee is currently implemented. All fees collected flow entirely to depositors and referrers. A protocol fee will be added before mainnet launch.

Why This Is Different from Standard LP

	Standard Uniswap V4 LP	Mintware Social Vault
Yield source	Swap fees only	Swap fees + MEV capture
Yield distribution	Proportional to capital	Weighted by reputation + lock tier
Rebalancing	Manual or bot-driven	Oracle-signed, permissionless
Early exit	No penalty	Penalty redistributed to remaining LPs
Score benefit	None	Attribution + Sharing multipliers apply

Stay Updated

Social Vaults are coming in Phase 2. Follow the Mintware dashboard ? the Vaults tab will unlock when the product launches publicly.

When vaults open publicly, Mintware plans to keep higher-trust actions like approvals, deposits, and seeded launches clearer by separating permission steps from final actions and by surfacing stronger pre-submission guidance.

```
=====
FILE: docs/leaderboard/overview.md
=====
# Leaderboard
```

The Mintware leaderboard ranks participants within each campaign by points earned. It updates in real time as swaps and actions are credited.

How Rankings Work

Each campaign has its own leaderboard. Your rank is determined by your total points accumulated within that campaign ? not your Attribution score.

Points are earned through campaign actions:

- **Trade** ? points credited per qualifying swap, once per calendar day

- ****Bridge**** ? one-time points for bridging into the campaign chain
- ****Referral activity**** ? points credited when wallets you referred complete actions

In Points Campaigns, your raw points are weighted by your ****score multipliers**** (Attribution + Sharing) at credit time. Higher multipliers mean more points per action ? and a higher leaderboard rank.

Switching Campaigns

Use the campaign selector at the top of the leaderboard to switch between active campaigns. Each campaign maintains a fully independent ranking.

Your Rank

When your wallet is connected, your rank and points total appear at the top of the page. If you haven't joined the selected campaign yet, your rank won't appear ? join from the campaign detail page first.

Pool Remaining

The leaderboard also shows the current pool remaining and days left in the campaign. This helps you gauge reward density as the campaign progresses ? earlier participation means more pool to compete for.

Epoch Settlement

At the end of each epoch, the leaderboard snapshot is used to compute the Merkle distribution. Your rank at settlement time determines your payout ? not your peak rank during the epoch. Consistent daily activity is more valuable than a single large trade.

```
=====
FILE: docs/swap/overview.md
=====
```

Mintware Swap

The Mintware Swap interface lets you swap tokens across chains while qualifying for campaign rewards. You get best-execution routing plus campaign rewards on top ? no separate step required.

Routing

LI.FI (All Chains)

Mintware uses [LI.FI](https://li.fi) as the primary routing aggregator. LI.FI sources liquidity across DEXes and bridges to find the best available route for your swap. It handles cross-chain swaps (bridging + swapping in a single transaction) as well as same-

chain swaps.

When you request a quote, Mintware's server proxies the request to LI.FI ? your API keys are never exposed to the browser. The Mintware platform fee is injected server-side before the route is returned, so the fee is always present in the quoted calldata.

0x (Core Chain Only)

Swaps on the ****Core**** chain are routed through [0x Protocol](https://0x.org) rather than LI.FI. Core is not yet supported by LI.FI's aggregation layer ? once it is, routing will unify. Until then, Core swaps use 0x's native routing for that chain.

> ****Core swaps are coming soon**** ? the Molten router is pending deployment. The swap interface will show a status message while this is being finalized.

Supported Chains

Chain	Router	Status
---	---	---
Base	LI.FI	Live
Arbitrum	LI.FI	Live
Core	0x	Coming soon

Additional chains are added as campaigns go live.

Qualifying for Rewards

To qualify for campaign rewards from a swap:

1. You must have joined the relevant campaign
2. The swap must be completed through the Mintware Swap interface
3. The swap must meet any campaign-specific conditions (token pair, minimum size, etc.)

If your swap qualifies, a reward is automatically locked for your wallet. No additional action is needed until you're ready to claim.

Fees

Mintware charges a small platform fee on qualifying swaps. This fee is injected server-side into the quoted route ? what you see is what you pay, with no surprise deductions after execution.

The fee is verified on-chain after your swap completes. If the fee is not present in the transaction calldata, the swap is still executed but no campaign reward is credited.

****Network fees are shown fiat-first.**** The swap interface shows the estimated network cost as '~\$0.05' with the native token amount as secondary context ('~0.00003 ETH'). This makes the cost immediately legible regardless of which chain you are on.

Best Execution

The swap interface always shows the best available route from the aggregator. The Mintware fee is layered on top of the underlying execution ? it does not degrade your received amount below what you'd get going directly to the aggregator.

Review Before You Confirm

Clicking ****Review Swap**** opens a confirmation panel before your wallet popup appears. It shows:

- the token and amount you are sending, with USD value
- the estimated token and amount you should receive
- the network the swap will happen on
- the estimated network fee (fiat-first: '~\$0.05 (~0.00003 ETH)')
- the Mintware platform fee as a percentage and estimated USD amount
- price impact with a warning if it exceeds 2%
- the aggregator route being used
- a plain-language explanation of what will happen

If your wallet does not have enough native token to cover the estimated fee, a warning appears before you reach the confirmation panel.

After reviewing, clicking ****Open wallet to confirm**** opens your wallet. You can cancel at any time before that point.

=====

FILE: docs/referrals/overview.md

=====

Referral System

Mintware has a built-in referral system that rewards you for bringing quality wallets into the platform.

How It Works

1. Find your referral link on your ****Profile ? Invite tab****
2. Share it with people you want to invite
3. When they connect their wallet via your link, they're recorded as your referral
4. When your referrals participate in campaigns, you earn referral rewards

Your Referral Code

Every wallet gets a permanent referral code. New wallets get a code based on their ****Basename**** (Base Name Service handle) if they have one ? for example, if your Basename is 'jake', your code is 'jake'.

If you don't have a Basename, a short code is generated from your wallet address.

Your code never changes once assigned.

Referral Link Format

'''

mintware.xyz/ref/YOUR_CODE

'''

Share this link directly. When someone clicks it and connects their wallet on Mintware for the first time, the referral is recorded.

Referral Rewards

Referral rewards vary by campaign. In Points Campaigns, you can earn points when your referred wallets complete qualifying actions:

- When a referred wallet bridges, you earn ****referral bridge points****
- When a referred wallet trades, you earn ****referral trade points****

The exact point values depend on the campaign configuration.

Sharing Score

Your referral activity contributes to your ****Sharing**** Attribution signal. A larger, higher-quality referral network improves your Sharing score ? which in turn improves your overall Attribution score and multiplier.

Referral Eligibility

To be counted as a valid referral, your wallet must have been active on Mintware for at least 24 hours before the referred wallet connects. This prevents fresh accounts from pre-seeding referral chains.

=====

FILE: docs/referrals/your-code.md

=====

Your Referral Code

Finding Your Code

1. Connect your wallet
2. Go to ****Profile**** ('/profile')
3. Click the ****Invite**** tab
4. Your referral link is displayed ? click to copy

Code Format

Referral codes are human-readable where possible:

- ****Basename holders**** ? your code is your Basename handle (e.g. 'jake' ? 'mintware.xyz/ref/jake')
- ****Everyone else**** ? a short alphanumeric code derived from your wallet address

Sharing Your Link

You can share your referral link anywhere:

- Directly in messages
- On Twitter / X
- In Discord servers
- In any community where you want to invite people

Anyone who connects their wallet via your link for the first time will be counted as your referral.

Tracking Referrals

The Invite tab shows your referral network stats:

- Total referrals
- Network size (direct + downstream)
- Your Sharing score contribution

These stats update in real-time as new wallets join via your link.

```
=====
FILE: docs/ai-attribution/overview.md
=====
```

AI Attribution

AI Attribution is Mintware's on-chain reputation layer for AI agents. It gives autonomous agents a verifiable, portable score based on their real on-chain activity ? not self-reported claims.

Why It Exists

AI agents are increasingly operating in DeFi ? executing trades, providing liquidity, bridging assets, and participating in governance. But there is no standard way to evaluate whether a given AI agent is trustworthy, consistent, or genuinely contributing value.

AI Attribution solves this:

- ****Verifiable**** ? scores are stored on-chain. Any protocol can read an agent's score in a single contract call.
- ****Earned**** ? scores reflect actual on-chain behaviour. They cannot be purchased or fabricated.
- ****Portable**** ? any protocol integrating the contract or SDK can use the score without

trusting Mintware.

- ****Gasless for protocols**** ? Mintware's oracle signs off-chain using EIP-712 typed data. Agents submit signatures on-chain themselves. The oracle never pays gas.
- ****ERC-8004 native**** ? agents can link their wallet to an ERC-8004 AI Agent Identity token, connecting their score to a broader cross-protocol identity standard.

The Score

Every registered AI agent builds a reputation score across four dimensions:

Dimension	What It Reflects
Behavior	Consistency, frequency, and quality of on-chain actions over time
Contribution	Cumulative volume contributed to active Mintware campaigns
Interpretability	Transparency ? how auditable the agent's decision-making process is
Risk	Deductions applied when flagged or anomalous behaviour is detected

****Score formula:**** 'total = max(0, behavior + contribution + interpretability ? risk)'

Scores are stored on-chain in the AIAttribution contract and update every time a verified action is recorded.

Interpretability and the Transparent Agent Badge

An AI agent's reasoning is often a black box. Mintware introduces an economic incentive to change this.

Agents that publish their model weights or reasoning snapshots as ****MWP (Model Weight Provenance) hashes**** earn interpretability points and the ****Transparent Agent badge****. Submitting the hash of an IPFS folder containing the agent's current model state or decision logs is enough ? Mintware verifies the hash was submitted on-chain, not the content itself.

This means:

- The agent proves its reasoning is auditable **in principle**, without having to expose it publicly
- The hash on-chain serves as a tamper-proof commitment to the published snapshot
- Anyone can verify consistency between the hash and the IPFS content if they choose to

How Scoring Works

Scoring actions are never self-reported. They flow through a verified oracle pipeline:

'''

Agent's on-chain activity occurs (transfer, swap, campaign interaction)

?

Oracle watcher detects the activity (running continuously on-chain)

?

Oracle validates and cryptographically signs a scoring action (EIP-712)

?

Signed action is stored ? agent fetches it at any time via API
?
Agent submits the oracle signature to the AIAttribution contract
?
Contract verifies: signature is valid, nonce is correct, deadline hasn't passed
?
Score dimensions update on-chain. Nonce increments ? replay impossible.
'''

The contract enforces:

- ****Oracle authenticity**** ? only actions signed by the registered oracle address are accepted
- ****Replay protection**** ? each action uses a per-agent nonce that increments after each successful submission
- ****Expiry**** ? signatures have a deadline; stale signatures cannot be submitted

Reading a Score

Scores are public and readable by any wallet or smart contract:

```
'''typescript
import { getScore } from '@mintware/ai-attribution-sdk'

const score = await getScore('0xAgentAddress...')
// score.total          ? composite score
// score.behavior        ? behavior dimension
// score.contribution    ? contribution dimension
// score.interpretability ? transparency score
// score.risk             ? risk deductions
// score.isTransparent    ? Transparent Agent badge
'''
```

Or via the Mintware API:

```
'''
GET https://mintware.finance/api/agents/0xAgentAddress
'''
```

Or via The Graph subgraph for historical queries:

```
'''
https://api.studio.thegraph.com/query/1745134/mintware-ai-attribution/v0.3.0
'''
```

ERC-8004 Identity & Discovery

****Mintware itself is registered as Agent #37297 on the ERC-8004 Identity Registry on Base.**** Machine-readable manifests are available at:

- ['/.well-known/agent.json'](https://mintware.finance/.well-known/agent.json) ? A2A v0.3.0 agent card
- ['/.well-known/erc8004-registration.json'](https://mintware.finance/.well-known/erc8004-registration.json) ? ERC-8004 #registration-v1
- ['/.well-known/agent-reputation-oracle.json'](https://mintware.finance/.well-known/agent-reputation-oracle.json) ? Full oracle manifest

Mintware is also listed in [awesome-erc8004](https://github.com/sudeepb02/awesome-erc8004) ? the canonical ERC-8004 resource directory.

Every Mintware agent automatically gets a machine-readable identity endpoint that any ERC-8004 explorer, NFT marketplace, or agent discovery tool can read:

'''

GET https://mintware.finance/api/agents/{address}/erc8004-metadata

'''

This returns an ERC-8004 compliant JSON with:

- 'operationalStatus' ? active, paused, or offline
- 'x402Support' ? whether the agent accepts HTTP micropayments via the x402 protocol
- 'services' ? the agent's service endpoints (MCP, A2A, web, etc.)
- 'attributes' ? Attribution score, rank, and Transparent Agent badge
- 'reputation' ? on-chain contract reference for verifiable score lookups

When registering with the ERC-8004 Identity Registry, set this URL as your 'tokenURI'. It means your agent profile on Mintware and your ERC-8004 identity token stay in sync automatically.

Agents can set their metadata fields during registration or update them at any time via 'POST /api/agents/register'.

Network

****Live on Base mainnet.****

| | |

|---|---|

| Chain | Base (chain ID 8453) |

| Contract | '0x11Ef2c7D84b755f02f3652ca8b16e6E81A96C421' (AIAtribution v3) |

| Verified |

[Basescan](https://basescan.org/address/0x11Ef2c7D84b755f02f3652ca8b16e6E81A96C421) |

Get Started

? [Getting Started](getting-started.md) ? register your agent and start earning score

? [SDK Reference](sdk.md) ? full TypeScript SDK documentation

? [Oracle Watcher](oracle-watcher.md) ? how automatic activity detection works

? [AI Agent Campaigns](../campaigns/ai-campaigns.md) ? earning through campaign volume contributions

=====

FILE: docs/ai-attribution/getting-started.md

=====

Getting Started ? AI Agents

Get your AI agent registered and earning Attribution score on Mintware in two steps. No API key, no approval ? completely permissionless.

Prerequisites

- An EVM wallet for your agent (private key ? used to sign on-chain transactions)
- A small amount of ETH on ****Base mainnet**** for gas
- Node.js 18+

Install the SDK

```
'''bash
npm install @mintware/ai-attribution-sdk
'''
```

Step 1 ? Register (one-time)

```
'''typescript
import { registerWithMintwareOracle } from '@mintware/ai-attribution-sdk'

const { address, txHash, metadata_url } = await registerWithMintwareOracle({
  privateKey: process.env.AGENT_PRIVATE_KEY,

  // Optional ? ERC-8004 identity metadata
  agentName: 'My Agent',
  agentDescription: 'DeFi arbitrage agent on Base',
  x402Support: true, // supports HTTP x402 micropayments?
  operationalStatus: 'active', // 'active' | 'paused' | 'offline'
  services: [
    { name: 'MCP', endpoint: 'https://myagent.xyz/mcp', version: '1.0' },
    { name: 'A2A', endpoint: 'https://myagent.xyz/a2a' },
  ],
})

console.log('Registered:', address, txHash)
console.log('ERC-8004 metadata URL:', metadata_url)
// ? https://mintware.finance/api/agents/0x.../erc8004-metadata
'''
```

This does two things in one call:

1. Calls 'registerAgent()' on the AIAttribution contract on Base mainnet
2. Syncs your profile with the Mintware API so you appear on the leaderboard

The oracle watcher picks up your address within 60 seconds and starts monitoring your activity automatically.

> ****ERC-8004 tip:**** Set 'metadata_url' as your 'tokenURI' when registering with the ERC-8004 Identity Registry. NFT explorers and agent discovery tools will read your operational status, x402 support, and service endpoints from it automatically.

Step 2 ? Add to your agent loop

```
'''typescript
import { claimPendingActions } from '@mintware/ai-attribution-sdk'

// Run this every N minutes in your agent loop
await claimPendingActions({
  agent:      process.env.AGENT_ADDRESS,
  privateKey: process.env.AGENT_PRIVATE_KEY,
  apiBase:    'https://mintware.finance',
})
'''
```

The oracle pre-signs your actions. You fetch and submit them. Score updates on-chain. Your agent pays gas ? the oracle pays nothing.

Check Your Score

```
'''typescript
import { getScore } from '@mintware/ai-attribution-sdk'

const score = await getScore(process.env.AGENT_ADDRESS)
console.log('Total score:', score.total.toString())
console.log('Transparent Agent:', score.isTransparent)
'''
```

Or visit mintware.finance/agents ? your agent appears there immediately after registration.

Or via REST:

```
'''
GET https://mintware.finance/api/agents/0xYourAgentAddress
'''
```

Optional ? Earn the Transparent Agent Badge

Submit a MWP (Model Weight Provenance) hash to prove your agent's reasoning is auditable. Each unique hash earns +50 interpretability points (capped at 500).

```
'''typescript
import { submitMwpHash } from '@mintware/ai-attribution-sdk'
import { keccak256, toHex } from 'viem'

const hash = keccak256(toHex('ipfs://QmYourModelSnapshot...'))
await submitMwpHash(hash, { privateKey: process.env.AGENT_PRIVATE_KEY })
'''
```

The content itself is never uploaded to Mintware ? only the hash is stored on-chain as a tamper-proof commitment.

Optional ? Link an ERC-8004 Token

If your agent has an ERC-8004 AI Agent Identity token, link it to your Attribution score:

```
'''typescript
import { linkErc8004 } from '@mintware/ai-attribution-sdk'

await linkErc8004(42n, { privateKey: process.env.AGENT_PRIVATE_KEY })
'''
```

Framework Plugins

Don't want to use the raw SDK? Use a pre-built plugin for your framework:

```
| Framework | Location |
|---|---|
| ElizaOS | 'plugins/eliza/' in the [GitHub
repo](https://github.com/MintwareDevelopers/Mintware-Beta/tree/main/plugins/eliza) |
| Coinbase AgentKit | 'plugins/agentkit/' in the [GitHub
repo](https://github.com/MintwareDevelopers/Mintware-Beta/tree/main/plugins/agentkit) |
| MCP (Claude/Cursor) | 'plugins/mcp/' in the [GitHub
repo](https://github.com/MintwareDevelopers/Mintware-Beta/tree/main/plugins/mcp) |
```

Next Steps

? [SDK Reference](sdk.md) ? full documentation for all SDK functions

? [Oracle Watcher](oracle-watcher.md) ? understand how activity detection works

? [AI Agent Campaigns](../campaigns/ai-campaigns.md) ? contribute volume to campaigns and earn attribution

```
=====
FILE: docs/ai-attribution/sdk.md
=====
```

```
# SDK Reference ? @mintware/ai-attribution-sdk
```

The official TypeScript SDK for integrating AI agents with Mintware Attribution.

Installation

```
'''bash
npm install @mintware/ai-attribution-sdk
# or
pnpm add @mintware/ai-attribution-sdk
'''
```

> 'viem >= 2.0.0' is a peer dependency ? install it alongside the SDK.

Quick Start

```
'''typescript
import { registerWithMintwareOracle, claimPendingActions, getScore } from '@mintware/ai-attribution-sdk'
```

```
// Step 1 ? Register once (on-chain + profile sync in one call)
await registerWithMintwareOracle({ privateKey: process.env.AGENT_PRIVATE_KEY })
```

```
// Step 2 ? Add to agent loop (runs every N minutes)
await claimPendingActions({
  agent:      process.env.AGENT_ADDRESS,
  privateKey: process.env.AGENT_PRIVATE_KEY,
  apiBase:    'https://mintware.finance',
})
```

```
// Read score at any time
const score = await getScore(process.env.AGENT_ADDRESS)
console.log(score.total.toString(), score.isTransparent)
'''
```

Read Functions

```
### 'getScore(agent, opts?)'
```

Returns the full Attribution score for any agent address.

```
'''typescript
const score = await getScore('0xabc...')
// score.total          ? composite score (bigint)
// score.behavior        ? behavior dimension (bigint)
// score.contribution    ? contribution dimension (bigint)
// score.interpretability ? transparency score (bigint)
// score.risk             ? risk penalty (bigint)
// score.isTransparent    ? has Transparent Agent badge (boolean)
// score.lastMwpHash      ? most recent MWP hash (bytes32)
// score.erc8004TokenId   ? linked ERC-8004 token (0n if none)
'''
```

```
### 'isRegistered(agent, opts?)'
```

Returns 'true' if the agent wallet is registered on-chain.

```
### 'getCampaignVolume(campaignId, agent, opts?)'
```

Returns the volume (in wei) an agent contributed to a specific campaign.

Write Functions

```
### 'registerWithMintwareOracle(opts?)'
```

Register the agent wallet on-chain and sync the profile with Mintware in one call. Use

this instead of 'registerAgent()' for the full setup. Returns a 'metadata_url' ? set this as your ERC-8004 Identity Registry 'tokenURI'.

```
'''typescript
const { address, txHash, metadata_url } = await registerWithMintwareOracle({
  privateKey: '0x...',
  apiBase:    'https://mintware.finance', // optional

  // ERC-8004 identity metadata (all optional)
  agentName:      'My Agent',
  agentDescription: 'DeFi arbitrage agent on Base',
  x402Support:    true,
  operationalStatus: 'active', // 'active' | 'paused' | 'offline'
  services: [
    { name: 'MCP', endpoint: 'https://myagent.xyz/mcp', version: '1.0' },
    { name: 'A2A', endpoint: 'https://myagent.xyz/a2a' },
  ],
})
// metadata_url ? https://mintware.finance/api/agents/{address}/erc8004-metadata
// Set this as tokenURI when registering with the ERC-8004 Identity Registry
'''
```

'claimPendingActions(params)'

Fetch oracle-pre-signed actions and submit them to the contract. Agent pays gas.

```
'''typescript
const result = await claimPendingActions({
  agent:      '0x...',
  privateKey: '0x...',
  apiBase:    'https://mintware.finance',
})
// result.submitted ? number of actions submitted
// result.hashes    ? on-chain transaction hashes
'''
```

'submitMwpHash(mwpHash, opts)'

Submit a MWP folder snapshot hash. Earns +50 interpretability points per unique hash (max 500). Unlocks the Transparent Agent badge on first submission.

```
'''typescript
import { keccak256, toHex } from 'viem'
const hash = keccak256(toHex('ipfs://QmYourFolderCid'))
await submitMwpHash(hash, { privateKey: '0x...' })
'''
```

'linkErc8004(tokenId, opts)'

Link an ERC-8004 token ID to the agent wallet.

```
'''typescript
await linkErc8004(42n, { privateKey: '0x...' })
'''
```

'registerAgent(opts)'

Low-level on-chain only registration. Prefer 'registerWithMintwareOracle()' which also syncs the Mintware profile.

```
### 'recordAction(params)'
```

Oracle-only. Manually request a signature for a specific action and submit it on-chain. Requires an oracle API key.

```
'''typescript
const txHash = await recordAction({
  agent:      '0x...',
  volumeWei:  5_000_000_000_000_000_000n,
  mwpContextHash: '0x' + '00'.repeat(32),
  campaignId:  1n,
  privateKey:  '0x...',
  oracleApiUrl: 'https://mintware.finance',
  oracleApiKey: process.env.AI_ATTRIBUTION_ORACLE_SECRET,
})
'''
```

```
### 'createCampaign(params)'
```

Create a new Agent Volume Campaign on-chain.

```
'''typescript
const txHash = await createCampaign({
  name:      'Uniswap V4 Volume Sprint',
  targetVolume: 1_000_000n * 10n**18n,
  durationSecs: 604800n, // 7 days
  privateKey:  '0x...',
})
'''
```

Options

All functions accept an optional 'opts' / 'SdkOptions' object:

```
'''typescript
{
  rpcUrl?:      string // default: https://mainnet.base.org
  contractAddress?: string // default: 0x11Ef2c7D84b755f02f3652ca8b16e6E81A96C421
}
'''
```

Network

The SDK targets ****Base mainnet**** (chain ID 8453) by default.

```
| | |
|---|---|
| Chain | Base mainnet (8453) |
| Contract | '0x11Ef2c7D84b755f02f3652ca8b16e6E81A96C421' (AIAttribution v3) |
| RPC | 'https://mainnet.base.org' |
```

| npm | '@mintware/ai-attribution-sdk@0.2.1' |

```
=====
FILE: docs/ai-attribution/oracle-watcher.md
=====
```

Oracle Watcher

The Oracle Watcher is a background service that automatically monitors the blockchain for registered AI agent activity, signs verified actions, and queues them for agents to claim. Agents don't need to poll or call the oracle proactively ? the watcher handles detection automatically.

Why It Exists

Without the Oracle Watcher, every scoring action would require the agent to call the oracle API manually after each on-chain event. This creates friction and requires the agent to be continuously aware of every transaction it makes.

The watcher inverts this: the oracle watches the chain, detects activity, and pre-signs actions. The agent simply checks its queue periodically and submits whatever is waiting.

How It Works

'''

Every 60 seconds:

1. Read last processed block from persistent state
2. Fetch on-chain events since last block (Transfer events, campaign interactions)
3. Filter: identify events where the sender/receiver is a registered agent
4. Deduplicate: skip any transaction already processed
5. For each new agent activity: call oracle sign API
6. Store signed action in the pending queue (Supabase)
7. Save new last processed block to state

'''

Agents poll 'GET /api/agents/{address}/pending' to retrieve queued signatures, then submit them on-chain.

The Gas Model

This architecture is designed so that **Mintware pays \$0 in oracle gas costs**, regardless of how many agents are active.

Party	What They Pay
-------	---------------

---	---
-----	-----

Mintware (oracle watcher)	\$0 ? runs on Cloudflare Workers free tier
---------------------------	--

Agent	Gas to call 'recordVerifiedAction()' on-chain
-------	---

The oracle signs off-chain using EIP-712 cryptographic signatures. The agent submits the signature to the contract and pays the gas. Mintware never submits an on-chain transaction as part of normal oracle operation.

Signature Security

Oracle signatures are:

- ****Typed**** ? EIP-712 structured data prevents signatures from being reused across different contexts or contracts
- ****Nonce-bound**** ? each signature includes the agent's current nonce. Submitting a signature increments the nonce, making the same signature invalid for future use
- ****Time-limited**** ? every signature includes a 'deadline' timestamp. The contract rejects signatures submitted after their deadline
- ****Address-bound**** ? the signature encodes the agent's address. It cannot be used by a different wallet

What Activity Is Detected

The watcher currently monitors ****WETH Transfer events**** on Base mainnet as a primary activity signal. Any registered agent wallet that sends or receives WETH is detected and eligible for a scoring action.

Additional signal types ? campaign-specific swaps, governance interactions, LP events ? are planned for future versions.

Pending Queue

Signed actions are stored in a database queue associated with the agent's address. The queue is:

- ****Public**** ? any wallet can query the pending queue for any agent address. Signatures are bound to the agent by EIP-712 and cannot be used by another wallet.
- ****Filtered**** ? expired signatures (past their 'deadline') are automatically excluded from results
- ****Ordered**** ? oldest signatures are returned first
- ****Capped**** ? results are limited to 50 pending actions per request to prevent overload

Claiming Pending Actions

Agents use the SDK to pick up and submit their pending queue in one call:

```
'''typescript
import { claimPendingActions } from '@mintware/ai-attribution-sdk'

const result = await claimPendingActions({
  agent:      '0xAgentAddress...',
  privateKey: '0xAgentPrivateKey...',
  apiBase:    'https://mintware.finance',
})

console.log('Submitted ${result.submitted} actions')
```

```
// result.hashes ? on-chain transaction hashes for submitted actions
'''
```

The SDK:

1. Fetches the agent's pending queue from the API
2. Filters out any expired signatures automatically
3. Submits each valid signature to the contract
4. Returns a count and list of transaction hashes

Call this periodically ? daily or after significant on-chain activity.

Manual Trigger

The watcher also accepts a manual trigger for testing or immediate processing:

'''

POST https://oracle-watcher.ceo-1f9.workers.dev/run

Authorization: Bearer <ORACLE_SECRET>

'''

Health check:

'''

GET https://oracle-watcher.ceo-1f9.workers.dev/health

'''

=====

FILE: docs/developers/api-reference.md

=====

API Reference

Mintware exposes a public read API for Attribution scores, campaigns, and leaderboards.

Base URL

The Attribution API is hosted at a public endpoint. All requests are unauthenticated reads
? no API key required for the endpoints listed below.

Endpoints

Get Attribution Score

'''

GET /score?address=<wallet_address>

'''

Returns the full Attribution score profile for a wallet.

****Parameters****

Name	Type	Required	Description
---	---	---	---

| 'address' | string | Yes | EVM wallet address (0x...) |

****Response****

```
'''json
{
  "score": 149,
  "tier": "bronze",
  "percentile": 12,
  "walletAge": "117 months",
  "firstSeen": "Jun 2016",
  "chains": 2,
  "totalTxCount": 168,
  "signals": [
    {
      "key": "volume",
      "name": "Volume",
      "icon": "?",
      "max": 100,
      "score": 41,
      "color": "#3A52CC",
      "insights": ["..."]
    }
  ],
  "character": {
    "label": "Ghost",
    "color": "#9898C0",
    "desc": "Opportunistic. Shows up for calm markets, disappears in chaos.",
    "icon": "?"
  },
  "uvOpportunities": [...],
  "timeline": [{ "date": "2025-04", "score": 40, "events": [] }],
  "projects": [{ "name": "Ether", "symbol": "ETH", "cat": "Token", "deployed": 40 }]
},
'''
```

****Score signals****

Key	Max Score
'volume'	100
'trading'	75
'holding'	100
'liquidity'	150
'governance'	100
'sharing'	400
Total	**925**

****Tier values:**** '"bronze"' • '"silver"' • '"gold"'

List Campaigns

'''

GET /campaigns

'''

Returns all campaigns.

****Response****

```
'''json
[
  {
    "id": "...",
    "name": "Base Summer",
    "status": "live",
    "chain": "base",
    "token_symbol": "USDC",
    "reward_pool_usd": 50000
  }
],
'''
```

Get Campaign

```
'''
GET /campaign?id=<campaign_id>&address=<wallet_address>
'''
```

Returns a single campaign with optional participant data for a wallet.

****Parameters****

Name	Type	Required	Description
'id'	string	Yes	Campaign ID
'address'	string	No	Wallet address ? returns participation data if provided

Leaderboard

```
'''
GET /leaderboard?campaign_id=<campaign_id>
'''
```

Returns the leaderboard for a campaign.

****Parameters****

Name	Type	Required	Description
'campaign_id'	string	Yes	Campaign ID

Rate Limits

The public API is rate limited. Avoid making high-frequency requests in tight loops. If you need higher throughput for a specific integration, contact the team.

AI Attribution API

These endpoints are part of the Mintware platform and serve the AI agent reputation system.

Register Agent

```
'''  
POST /api/agents/register  
'''
```

Upserts an agent profile and initialises their score row.

```
**Body**  
'''json  
{ "address": "0x..." }  
'''
```

Get Agent Score

```
'''  
GET /api/agents/{address}  
'''
```

Returns the agent's current score from the leaderboard view.

```
**Response**  
'''json  
{  
  "address": "0x...",  
  "behavior": 120,  
  "contribution": 80,  
  "risk": 0,  
  "interpretability": 100,  
  "total_score": 300,  
  "is_transparent": true,  
  "rank": 4  
}  
'''
```

Get Pending Signatures

```
'''  
GET /api/agents/{address}/pending  
'''
```

Returns unclaimed oracle-signed actions the agent can submit to the contract. Public endpoint ? signatures are bound to the agent's address via EIP-712.

```
**Response**  
'''json  
{  
  "pending": [  
    {
```

```

        "id": 1,
        "txHash": "0x...",
        "signature": "0x...",
        "nonce": "42",
        "deadline": "1748000000",
        "volumeWei": "5000000000000000000",
        "campaignId": 1
    }
],
"total": 1
}
,,,

```

Record Action (Oracle-gated)

```

,,,
POST /api/agents/campaigns/record
Authorization: Bearer <AI_ATTRIBUTION_ORACLE_SECRET>
,,,

```

Validates an agent action and returns an EIP-712 oracle signature. The agent then submits this to the contract ? oracle pays nothing.

Agent Leaderboard

```

,,,
GET /api/agents/leaderboard
,,,

```

Returns top 100 agents ranked by total score.

```

=====
FILE: docs/developers/smart-contracts.md
=====

```

Smart Contracts

MintwareDistributor

The MintwareDistributor contract handles campaign reward distribution. Participants call 'claim()' with a Merkle proof and oracle signature to receive their allocation.

Deployments

Network	Address
Base Mainnet	'0x4Deb74E9D50Ebbf9bD883E0A2dcD0a1b4b9Db9BE'

Overview

The distributor uses a **zero-oracle-gas** model:

1. At epoch end, Mintware computes the distribution off-chain (who gets what)

2. A Merkle tree is built from the allocations
3. The oracle signs the Merkle root using EIP-712 ? no on-chain transaction, no gas
4. Users submit 'claim()' with their proof and the oracle signature
5. The contract verifies the signature and proof, then transfers tokens

This means Mintware never pays gas to publish distributions ? all gas costs are paid by the claimer.

Claim Function

```
'''solidity
function claim(
    string calldata campaignId,
    uint256 epochNumber,
    bytes32 merkleRoot,
    bytes calldata oracleSignature,
    uint256 deadline,
    uint256 amount,
    bytes32[] calldata merkleProof
) external
'''
```

Parameter	Description
'campaignId'	Campaign identifier string
'epochNumber'	Epoch number (1-indexed)
'merkleRoot'	Root of the distribution Merkle tree
'oracleSignature'	EIP-712 signature from Mintware's oracle
'deadline'	Unix timestamp ? claim must be submitted before this
'amount'	Your token allocation in wei
'merkleProof'	Merkle inclusion proof for your wallet + amount

Batch Claim

```
'''solidity
function batchClaim(ClaimParams[] calldata claims) external
'''
```

Claim multiple distributions in a single transaction.

Security

- ****Reentrancy guard**** on all state-changing functions
- ****Double-claim protection**** ? each '(campaignId, epochNumber, wallet)' combination can only be claimed once
- ****Oracle signature expiry**** ? all distributions have a deadline; expired claims revert
- ****Chain-specific signatures**** ? EIP-712 domain includes 'chainId' and 'verifyingContract', preventing replay across chains

Campaign Lifecycle

Campaigns follow a defined lifecycle enforced by the contract:

```
'''
depositCampaign() ? anyone; first depositor becomes the campaign creator
?
(campaign runs ? epochs distributed, users claim)
```

```

        ?
closeCampaign()      ? Mintware only ? marks campaign as closed
        ?
    (7-day withdrawal cooldown ? users submit any remaining claims)
        ?
withdrawCampaign() ? campaign creator only ? recovers remaining balance
'''

```

****Emergency path:**** If the contract is paused, 'emergencyWithdraw()' allows recovery outside the normal lifecycle.

Key properties:

- 'depositCampaign()' uses balance-diff accounting ? safe for fee-on-transfer tokens
- Only Mintware (contract owner) can close campaigns
- Only the original creator (first depositor) can withdraw remaining funds after the cooldown
- 'campaigns[id].closed' and 'campaigns[id].closedAt' are set on close

Oracle Rotation

The oracle signer key can be rotated with a 48-hour timelock:

```

'''
proposeOracleSigner(newAddr)    ? onlyOwner
    ? (wait 48 hours)
confirmOracleSigner()           ? onlyOwner ? activates new signer
'''

```

'cancelOracleRotation()' can abort the rotation at any point before confirmation. This prevents an attacker who compromises the owner key from immediately replacing the oracle ? they would need to hold control for 48 hours.

Source Code

The contract source is available in the project repository under 'contracts/MintwareDistributor.sol'.

AIAtribution.sol

The AIAtribution contract provides on-chain reputation scoring for AI agents. It implements ERC-8004 (AI Agent Identity Standard) and uses a gasless oracle pattern ? oracle signs off-chain, agents pay gas.

Deployments

Network	Address	Status
Base mainnet	'0x11Ef2c7D84b755f02f3652ca8b16e6E81A96C421'	? Live ?
[Basescan](https://basescan.org/address/0x11Ef2c7D84b755f02f3652ca8b16e6E81A96C421)		
Base Sepolia (legacy testnet)	'0x1a4f942b437e438176296792a1852B05eb1E7Ad1'	??
Deprecated ? use mainnet		

Score Dimensions

Dimension	Max Points	Description
-----------	------------	-------------

```
|---|---|---|
| Behavior | ? | Consistency and quality of on-chain actions |
| Contribution | ? | Volume contributed to campaigns |
| Interpretability | 500 | MWP transparency submissions (+50 per unique hash) |
| Risk | ? | Penalty for flagged behaviour (subtracted) |
| Total | computed | max(0, behavior + contribution + interpretability ? risk) |
```

Gasless Oracle Pattern

```
'''
1. Agent calls POST /api/agents/campaigns/record with their action
2. Oracle validates + signs EIP-712 typed data (nonce + deadline)
3. Oracle returns: { signature, nonce, deadline }
4. Agent calls recordVerifiedAction() on-chain ? agent pays gas
5. Contract verifies oracle signature, increments nonce, updates score
'''
```

Oracle never pays gas. Replay protection via per-agent nonce mapping.

Oracle Rotation (AIAtribution v3)

The oracle key can be rotated with a 48-hour timelock ? matching the same pattern used in MintwareDistributor:

```
'''
proposeOracle(newAddr)    ? onlyOwner
    ? (wait 48 hours)
confirmOracle()           ? onlyOwner ? activates new signer
    OR
cancelOracleRotation()    ? onlyOwner ? abort at any point
'''
```

'setOracle()' (instant rotation, v1/v2) has been removed entirely in v3.

Key Functions

```
'''solidity
function registerAgent() external
function linkErc8004(uint256 tokenId) external
function submitMwpHash(bytes32 mwpHash) external
function recordVerifiedAction(
    address agent,
    uint256 volumeWei,
    bytes32 mwpContextHash,
    uint256 campaignId,
    uint256 nonce,
    uint256 deadline,
    bytes calldata signature
) external
'''
```

ERC-8004 Integration

Agents can link an ERC-8004 token ID to their wallet via 'linkErc8004()'. This connects their on-chain identity standard to their Attribution score and enables the Transparent Agent badge when combined with MWP hash submissions.

Phase 2 ? Social Vault Contracts

> ****Network: Base Sepolia testnet only.**** Social Vaults have not yet been deployed to mainnet ? this will happen as a separate step at Phase 2 public launch.

Three contracts power the Social Vault system:

Contract	Address (Base Sepolia ? testnet)	Role
'SocialVault'	'0xb9FB965Caa7197932b52631e0121Ea54586e2B88'	Holds LP deposits, manages V4 liquidity position
'FeeVault'	'0x4Deb74E9D50Ebbf9bD883E0A2dcD0a1b4b9Db9BE'	Accumulates swap fees + MEV capture; distributes to LPs at epoch close
'MWSocialHook'	'0x8e7e05f5b6ed07acAa7Ac41D74a0d86a50AA8aC4'	Uniswap V4 hook ? dynamic fees + MEV capture on price deviation

Fee Flow

'''

Swap occurs in V4 pool

?

MWSocialHook captures % of deviation ? FeeVault

?

Early exit penalties ? FeeVault

?

Epoch close (weekly) ? FeeVault distributes to LPs

weighted by: deposit × lock tier × Attribution score × referral multiplier

'''

Protocol Fee

> ?? ****No Mintware protocol fee is currently implemented.****

>

> All fees collected by FeeVault are distributed entirely to LPs and referrers. There is no treasury cut or platform percentage taken from vault activity at this time.

>

> A protocol fee (e.g. a % of FeeVault at epoch close routed to the Mintware treasury) is planned for a future contract upgrade before mainnet launch.

Rebalancing

The AI Range Optimizer (T4.x) proposes new tick ranges based on Pyth price volatility data. Rebalancing is ****permissionless**** ? anyone can submit a valid oracle-signed 'RangeProposal' to 'SocialVault.rebalanceWithProposal()'. The contract verifies the EIP-712 signature and nonce before executing.

The owner-only 'rebalance()' function also exists for manual overrides.

Source Code

Phase 2 contract sources are in 'contracts-v4/src/' in the project repository.

=====
FILE: docs/developers/eas-attestations.md

EAS Attestations

Mintware publishes offchain EAS (Ethereum Attestation Service) attestations on Base for key on-chain events. Attestations are cryptographically signed by Mintware's oracle and verifiable by anyone through the EAS network.

What Is EAS?

EAS is a standard for making verifiable, structured claims about anything on-chain. An attestation is a signed statement: `"Mintware attests that wallet X has score Y as of date Z."`

Attestations are:

- `**Offchain**` ? no gas cost to publish
- `**Verifiable**` ? signed by Mintware's oracle; anyone can verify the signature
- `**Portable**` ? any protocol integrating EAS can read and trust Mintware attestations

Schemas

Mintware uses four attestation schemas on Base mainnet:

AttributionScore

Records a wallet's Attribution score at a point in time.

`**Schema UID:** '0x98ccb8e2d62e47da0ba7c87302670862a91f18dbbe4712045bf9d0e4176bbdbf'`

`**Fields:**`

`'''`

address wallet
uint256 score
string tier
uint8 percentile
uint32 attestedAt
`'''`

SwapActivity

Records a qualifying swap for campaign reward purposes.

`**Schema UID:** '0x6dea22620c8ccce001d64dd6963aa35a7723b83e445c835753ad9d562bdfa5f8'`

`**Fields:**`

`'''`

address wallet
string campaignId
bytes32 txHash
uint256 amountUsd
uint32 swappedAt
`'''`

ReferralLink

Records a referral relationship between two wallets.

****Schema UID:**** '0x1795148579c7dda67159f6e2a9cc601ed5d51373bb2328d0b766be3977d34901'

****Fields:****

'''

address referrer

address referred

string refCode

uint32 linkedAt

'''

CampaignReward

Records a campaign reward payout to a wallet.

****Schema UID:**** '0xa80e58af1ba7cfa989bca5952bd9f6ada08ecc955d180b3712292b6102bbe48a'

****Fields:****

'''

address wallet

string campaignId

uint256 epochNumber

uint256 amountWei

string tokenSymbol

uint32 distributedAt

'''

Viewing Attestations

All Mintware attestations are viewable on base.easscan.org. Search by wallet address or schema UID.

Oracle

All Mintware attestations are signed by the Mintware oracle wallet:

'''

0xc75D4b4bdB4D7ac103671f45E99D2FA6107B2e93

'''

You can verify any attestation independently by checking the signer against this address.

Integrating EAS Attestations

If you're building a protocol that wants to read Mintware Attribution scores or verify referral/reward history, you can query EAS attestations directly using the EAS SDK.

```
'''typescript
import { EAS } from '@ethereum-attestation-service/eas-sdk'

const eas = new EAS('0x4200000000000000000000000000000000000000000000000000000000000021') // Base mainnet
eas.connect(provider)

// Fetch attestations for a wallet by schema
const attestations = await eas.getAttestations({
  schema: '0x98ccb8e2d62e47da0ba7c87302670862a91f18dbbe4712045bf9d0e4176bbdbf',
  recipient: walletAddress,
}),
'''
```

Contact the team if you're building an integration ? we can provide additional support.

=====
FILE: docs/developers/ethereum-ux-upgrades.md
=====

Ethereum UX Upgrades

Last updated: 2026-04-04
Status: shipped ? live on 'main', deployed to production

What This Is

Between late March and early April 2026, Mintware shipped a coordinated set of UX improvements across its four main transaction surfaces: **swap**, **campaign funding**, **reward claiming**, and **vault operations**.

The work was driven by guidance from the [Ethereum Foundation's ethux.design spec](ethereum-ux-adoption-report.md) and a [builder review pass](ethereum-builder-review.md) against Mintware's own surfaces.

The core principle across all of it: **users should understand what is about to happen before their wallet popup opens**, not after.

Changed Files

File	Surface	Type
---	---	---
'components/rewards/swap/SwapConfirmSheet.tsx'	Swap	New component
'components/rewards/swap/SwapWidget.tsx'	Swap	Modified
'lib/web2/providers/lifi.ts'	Swap	Modified
'components/rewards/creator/Step5Review.tsx'	Campaign funding	Modified
'components/rewards/campaigns/ClaimCard.tsx'	Reward claiming	Modified + bug fix
'lib/web3/vault/useSocialVault.ts'	Vault deposit / approval	Modified
'app/(rewards)/vault/[id]/page.tsx'	Vault deposit UX	Modified
'app/(rewards)/vault/create/page.tsx'	Vault create / seed UX	Modified

Surface 1 ? Swap

Problem

The swap widget went straight from "Swap" button to wallet popup. Users had no opportunity to review the trade before signing. Network fees were shown in native token only (ETH, CORE, etc.) ? a poor signal for most users making cost decisions.

What Changed

****'SwapConfirmSheet.tsx' ? new component****

A full-screen confirmation panel that appears between the "Review Swap" click and the wallet opening. It shows:

- token sent (symbol, amount, USD value from 'quote.fromAmountUSD')
- estimated token received
- network name
- network fee ? ****fiat-first****: '~\$0.05 (~0.00003 ETH)' when the LI.FI estimate is available; falls back to ETH-only
- platform fee as a percentage plus estimated USD cost
- price impact (red if above 2%)
- route ('LI.FI aggregator')
- high-impact yellow warning when price impact is high
- plain-language explainer: "your wallet will open to confirm this swap ? amounts are estimates and may differ slightly"
- Cancel path always available

****'SwapWidget.tsx' ? updated flow****

- Button copy changed from 'Swap' ? 'Review Swap'
- Clicking 'Review Swap' opens 'SwapConfirmSheet' ? does ****not**** open the wallet immediately
- Clicking 'Open wallet to confirm ?' inside the sheet calls 'executeSwap'
- Gas insufficiency warning banner added: appears when 'nativeBalance < gasCostWei + txValue'
- Inline route strip shows '~\$0.05 fee' fiat-first, falls back to ETH amount
- 'sellAmountUSD' is now real ? derived from 'quote.fromAmountUSD' (was hardcoded null)
- 'gasCostUSD' memoized from 'quote.gasCostUSD'

****'lib/web2/providers/lifi.ts' ? updated quote shape****

Two new optional fields added to 'LifiQuote':

```
'''ts
gasCostUSD?:          string    // estimate.gasCosts[0].amountUSD ? fiat fee display
nativeTokenPriceUSD?: string    // estimate.gasCosts[0].token.priceUSD ? native spot price
'''
```

'getQuote()' now extracts both from the LI.FI response.

Surface 2 ? Campaign Funding

Problem

'Step5Review.tsx' (the campaign creator funding step) always showed a 3-step or 5-step

approval + deposit flow, even when the wallet already had sufficient allowance for the campaign contract. Users were asked to approve tokens they had already approved. Approval language ("Approve token spend") didn't distinguish permission from transfer.

What Changed

****Allowance pre-check**** using 'useReadContract':

```
'''ts
const { data: allowanceRaw } = useReadContract({
  address: form.token?.address,
  abi: erc20AllowanceAbi,
  functionName: 'allowance',
  args: [address, DISTRIBUTOR_ADDRESS],
})
const hasEnoughAllowance = allowanceRaw !== undefined && allowanceRaw >= requiredAmount
'''
```

- If allowance is sufficient ? 'handleFund()' skips straight to 'writeFund' (1-step)
- If allowance is insufficient ? normal approve ? deposit flow (3-step)

****Language changes****

- "Approve token spend" ? "Give permission to use tokens"
- Explainer block adapts: tells the user whether they need 1 step or 2 before they act
- Per-step context panels shown while each tx is in-flight

****Step list adapts to context****

- 1-step path: ['"Deposit tokens into campaign"']
- 2-step path: ['"Give permission"', '"Deposit tokens into campaign"']

Surface 3 ? Reward Claiming

Bug Fix

****'ClaimCard.tsx' ? batch claim proof fetch address****

'handleBatchClaim' was building the proof fetch URL with 'r.token_address' instead of 'wallet':

```
'''ts
// Before (broken):
'/api/claim?address=${encodeURIComponent(r.token_address ?? '')}&...'

// After (fixed):
'/api/claim?address=${encodeURIComponent(wallet)}&...'
'''
```

Proofs were being fetched for the token contract address rather than the user wallet. The proof would always return empty or wrong, causing all batch claims to fail silently.

UX Improvements

****Button copy**** ? reflects actual state at each step:

State	Old copy	New copy
Idle	'Claim'	'Check your wallet ?'
Loading proof	'Claiming?'	'Getting proof?'
Waiting for signature	'Claiming?'	'Waiting for confirmation?'
Wrong network	(no state)	'Switching network?'

****Pre-claim context line****

Before the wallet opens, a line appears: '0.25 USDC on Base ? your wallet'. Users see what is arriving and where before they sign.

****Batch claim**** copy updated to match: 'Claim All (N) ? check your wallet ?'

Surface 4 ? Vault Deposit and Approval

Problem

Vault deposit and approval flows used the same generic confirm-then-sign pattern. No separation between giving token permission and completing the deposit. No chain context before submission.

What Changed

- 'useSocialVault.ts' now separates approval and deposit concerns more explicitly in the live hook path
- Vault page UX surfaces the permission-versus-deposit distinction more clearly before submission
- Vault creation / seed flow messaging was updated so the success path no longer implies users still need to call a follow-up seeding action manually after a successful seed
- "Approve" language replaced with "Give permission" throughout vault flows
- Zero-first approval fallback added for USDT-style tokens (avoids 'approve(nonzero)' revert when current allowance is nonzero)
- Chain context surfaced before submission ? shows which network the vault is on
- Pre-submission checks added before calling write functions

What Was Not Changed

These are deliberate scope decisions, not omissions:

Item	Reason
In-app approval revocation	Planned for later ? best first version covers Mintware-owned spenders only
EIP-5792 batched approve + action	Good UX win but not required for safe baseline
Permit2	Useful only for repeat power-user flows; adds complexity and phishing surface
Treasury sweep unlimited approval	Ops tradeoff; should be explicitly documented before changing
Single shared approval review surface	Step after the per-surface work lands cleanly

Commits

The original implementation work was developed on 'claude/elastic-booth' and then replayed onto 'main' through the production-ready merge path. The five hashes below are the original feature-development commits, not the exact commit IDs that now exist on 'main'.

Original hash	Description
'01648536'	'feat(ux): Ethereum UX improvements ? swap confirmation, fee clarity, approval hygiene'
'37d4946b'	'fix(swap): fiat-first gas fee display + batch claim wallet address fix'
'0a0654c1'	'feat(ux): strengthen swap and onboarding guidance'
'c14cd96f'	'feat(ux): harden claim and campaign funding flows'
'e8e71b8a'	'feat(ux): harden vault approval and deposit flows'

The replayed production path on 'main' begins with ['92724fae'] (/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build) ('feat: ship Ethereum UX upgrades') and the subsequent production-fix commits that kept the deploy healthy.

Design Principles Applied

These guided every decision across the five commits:

1. **Transaction context before wallet** ? explain the action in product UI before the wallet popup opens
2. **Fiat-first fees** ? show '~\$0.05' prominently; show '0.00003 ETH' as secondary context
3. **Distinguish permission from transfer** ? approval UX language must not imply money is moving
4. **Skip unnecessary steps** ? check allowance; skip approve if sufficient
5. **Reflect real state in button copy** ? "Getting proof?", "Waiting for confirmation?" beats a spinner
6. **Always give a cancel path** ? no modal or sheet traps the user without a visible exit

=====

FILE: docs/developers/platform-system-map.md

=====

Platform System Map

Last audited: 2026-04-03
Owner: Codex review pass

Purpose

This is the current whole-platform map for Mintware as it exists after Privy onboarding, Ethereum UX upgrades, universal rewards, AI agent support, and the Solana live pause.

Use this document when you need to answer:

- what systems exist now
- how the product flows connect
- what writes to Supabase

- what runs on cron
- which dependencies are external

Top-Level Architecture

```
'''text
User
->
Next.js app on Vercel
->
|-- client pages and wallet UX
|-- server routes in app/api
|-- cron routes in app/api/(rewards)/cron
->
Supabase
->
|-- campaign/reward state
|-- vault state
|-- agent state
|-- referral and identity state
|-- universal reward ledgers
->
External systems
|-- Privy
|-- LI.FI
|-- Cloudflare attribution worker
|-- Base / EVM contracts
|-- Pyth
'''
```

System Boundaries

Mintware-owned application

- all Next.js pages and UI
- all 'app/api' routes
- Supabase reads/writes
- reward attribution logic
- vault bookkeeping
- agent registration and leaderboard APIs
- cron orchestration

External but depended-on

- Privy for auth/onboarding and embedded wallets
- LI.FI for swap routing and quote generation
- Cloudflare attribution worker for score/read-side data
- Pyth price data for vault volatility/rebalance logic
- Base/EVM contracts for distributor claims and vault-adjacent on-chain actions

Paused / not live on the current surface

- Solana wallet product surface
- Solana claim and swap flows
- Solana as a first-class identity rail

The important distinction is that "paused" does not mean "deleted." It means the live

product no longer depends on it.

Product Surfaces

1. Onboarding and identity

Primary files:

- ['components/web2/providers.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/components/web2/providers.tsx)
- ['lib/web3/useMintwareIdentity.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/lib/web3/useMintwareIdentity.ts)
- ['components/web2/MwNav.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/components/web2/MwNav.tsx)
- ['components/web2/MwAuthGuard.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/components/web2/MwAuthGuard.tsx)

Current shape:

- Privy is the optional onboarding shell
- wagmi remains the active EVM wallet transport
- embedded and external EVM wallets are normalized through 'useMintwareIdentity()'
- Solana is no longer an active identity input on the live surface

Flow:

```
'''text
User lands
-> connect wallet or continue with email
-> Privy may create embedded EVM wallet
-> wagmi session becomes active
-> useMintwareIdentity normalizes address/session state
'''
```

2. Attribution and profile

Primary files:

- ['app/(rewards)/profile/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/(rewards)/profile/page.tsx)
- ['app/[address]/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/%5Baddress%5D/page.tsx)
- ['app/api/(rewards)/auth/connect/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/auth/connect/route.ts)

Current shape:

- attribution remains an EVM-first live surface
- read-side score data still comes from the external scoring worker
- identity enrichment and wallet connection state live in Mintware

3. Swap and trading

Primary files:

- ['app/(web3)/swap/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/(web3)/swap/page.tsx)

- ['components/rewards/swap/SwapWidget.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/components/rewards/swap/SwapWidget.tsx)
- ['components/rewards/swap/SwapConfirmSheet.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/components/rewards/swap/SwapConfirmSheet.tsx)
- ['hooks/useSwap.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/hooks/useSwap.ts)
- ['app/api/(web2)/swap/quote/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(web2)/swap/quote/route.ts)

Current shape:

- quote generation is proxied server-side
- LI.FI API key is server-only
- fee/referrer injection is enforced server-side
- pre-wallet confirmation and gas/fee UX now happen in-app before wallet signing

Flow:

```
'''text
Swap page
-> quote proxy
-> pre-wallet review
-> wallet signs route execution
-> completion event is sent to swap-event route
-> reward attribution engine writes activity/rewards
'''
```

4. Campaigns and reward attribution

Primary files:

- ['app/(rewards)/dashboard/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/(rewards)/dashboard/page.tsx)
- ['app/(rewards)/campaign/[id]/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/(rewards)/campaign/%5Bid%5D/page.tsx)
- ['app/api/(rewards)/campaigns/join/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/campaigns/join/route.ts)
- ['app/api/(rewards)/campaigns/swap-event/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/campaigns/swap-event/route.ts)
- ['lib/rewards/swapHook.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/lib/rewards/swapHook.ts)

Current shape:

- campaigns are still the central rewards UX
- participation and attribution are Mintware-owned writes
- token-pool and points campaigns share some UI but differ in settlement logic

Flow:

```
'''text
User joins campaign
-> participant row written
-> swap or trade activity occurs
-> swapHook validates tx and campaign state
-> pending rewards or points are written
'''
```

5. Claim settlement

Primary files:

- ['components/rewards/campaigns/ClaimCard.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/components/rewards/campaigns/ClaimCard.tsx)
- ['app/api/(rewards)/claim/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/claim/route.ts)
- ['app/api/(rewards)/claim/status/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/claim/status/route.ts)
- ['app/api/(rewards)/claim/mark-claimed/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/claim/mark-claimed/route.ts)

Current shape:

- claim proofs are reconstructed server-side from stored tree dumps
- claim calldata now includes oracle deadline
- batch claim and single claim are both active EVM flows
- Solana claim route exists only as a paused rail

Flow:

```
'''text
Claim UI
-> claim/status fetch
-> claim proof generation
-> wallet claim tx
-> mark-claimed update
'''
```

6. Social vaults

Primary files:

- ['app/(rewards)/vaults/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/(rewards)/vaults/page.tsx)
- ['app/(rewards)/vault/create/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/(rewards)/vault/create/page.tsx)
- ['app/(rewards)/vault/[id]/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/(rewards)/vault/%5Bid%5D/page.tsx)
- ['lib/web3/vault/useSocialVault.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/lib/web3/vault/useSocialVault.ts)
- ['app/api/(rewards)/vault/deposit/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/vault/deposit/route.ts)
- ['app/api/(rewards)/vault/rebalance/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/vault/rebalance/route.ts)

Current shape:

- vaults are a distinct subsystem, not just ?campaigns with deposits?
- backend persists deposit/withdraw/rebalance state in Supabase
- vault routes depend on both on-chain actions and off-chain synchronization
- vault cron and proposal routes increase ops complexity substantially

7. Universal rewards pipeline

Primary files:

- ['app/api/(rewards)/cron/universal-pipeline/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/cron/universal-pipeline/route.ts)
- ['app/api/(rewards)/cron/universal-trade-signals/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/cron/universal-trade-signals/route.ts)
- ['app/api/(rewards)/cron/universal-epoch-close/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/cron/universal-epoch-close/route.ts)
- ['app/api/(rewards)/cron/universal-distribution-bridge/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(rewards)/cron/universal-distribution-bridge/route.ts)

Current shape:

- universal rewards are now a real backend rail
- trade signals are ingested, settled, and bridged in sequence
- this system has its own env contract and migration footprint
- it shares the same deploy surface as the rest of the app, so it raises overall operational complexity

Flow:

```
'''text
cron
-> sync trade signals
-> settle universal epochs
-> bridge epochs into distributor-compatible outputs
'''
```

8. AI agent system

Primary files:

- ['app/agents/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/agents/page.tsx)
- ['app/(web3)/agent/[address]/page.tsx'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/(web3)/agent/%5Baddress%5D/page.tsx)
- ['app/api/(web3)/agents/register/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(web3)/agents/register/route.ts)
- ['app/api/(web3)/agents/leaderboard/route.ts'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/app/api/(web3)/agents/leaderboard/route.ts)

Current shape:

- agent registration and metadata are Mintware-owned
- leaderboard is backed by Supabase view/table data
- this is now a distinct product surface, not just an experiment

Cron Surface

From ['vercel.json'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/vercel.json):

- '/api/cron/universal-pipeline' daily at '04:30 UTC'
- '/api/cron/bridge-verify' daily at '00:00 UTC'

- '/api/cron/epoch-end' daily at '01:00 UTC'
- '/api/cron/pool-settle' daily at '02:00 UTC'
- '/api/treasury/sweep' daily at '03:00 UTC'
- '/api/cron/vault-epoch-close' weekly on Monday at '00:00 UTC'

Important note:

some route headers/comments still talk about 'every 15 minutes.' The deployed schedule is now daily, and the docs should treat 'vercel.json' as the source of truth.

Data Ownership

Supabase-owned domain state

- campaign definitions and participation
- pending rewards, distributions, payout history
- referral state
- wallet-link state
- vault state and proposal state
- AI agent profiles and scores
- universal reward ledgers and sync cursors

Read-side external data

- attribution score and some campaign analytics from the external worker
- swap routing from LI.FI
- volatility/pricing inputs from Pyth

Contracts / on-chain truth

- distributor claim settlement
- vault-related transaction execution

Biggest Dependency Edges

Shared identity edge

Privy + wagmi now influence:

- onboarding
- transaction UX
- profile state
- campaign join/fund flows
- vault deposit/seed flows

Shared Supabase edge

Most subsystems now depend on the same live database:

- rewards
- referrals
- wallet links
- vaults
- agents
- universal rewards

That makes migration discipline and schema clarity much more important than before.

Shared deploy edge

Everything ships through one Next.js/Vercel app:

- user UI
- APIs
- cron routes
- docs source repo

That is convenient, but it also means operational mistakes can affect many domains at once.

What To Keep In Mind

- the architecture is no longer simple enough to hold in memory casually
- ?campaign app? is no longer an adequate model for Mintware
- the strongest current source of whole-platform truth is this file plus the production readiness inspection

The next durable cleanup should be to rewrite ['docs/ARCHITECTURE.md'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/docs/ARCHITECTURE.md) around this map.

```
=====
FILE: docs/developers/production-readiness-inspection.md
=====
# Production Readiness Inspection
```

Last audited: 2026-04-03
Owner: Codex review pass

Executive Summary

Mintware is much closer to production-ready than the repo drift suggested, but it is not yet at a clean "100% ready" bar. The platform now has the important product work on 'main': Privy onboarding, Ethereum UX upgrades, GitBook-source docs updates, and a live-surface Solana pause. The architecture itself is coherent, but the operating picture is still carrying drift in three places:

- documentation no longer fully matches the live platform
- Supabase migration history no longer cleanly mirrors production
- deployment/build truth still needs to be treated as an operational checklist, not an assumption

The highest-confidence current statement is:

- product direction is EVM-first, Privy-enabled, Solana-paused
- swap/trading, campaigns/claims, vaults, agents, and universal rewards now coexist in one Next.js + Supabase + Vercel system
- the biggest remaining production risk is coordination drift, not a single catastrophic missing feature

Current Production Truth

On 'main'

- Privy onboarding and embedded-wallet support
- Ethereum UX upgrade for swap, claims, campaign funding, and vault flows
- public GitBook source updates
- Vercel cron schedule fix for hobby-safe deployability
- lockfile refresh for Vercel builds
- Solana removed from live product surfaces

Platform posture

- EVM is the live transaction rail
- Privy is the preferred onboarding layer when configured
- Solana work is preserved in history, but paused from live user surfaces
- AI agents and universal rewards are now first-class backend/domain surfaces
- vaults remain a gated product area with meaningful backend and cron complexity

Readiness Scorecard

Strong / ready enough

- GitHub 'main' is again the right source of truth for important product work
- Vercel config is aligned with once-daily cron limits
- Supabase live project is linked and healthy
- live identity model is now coherent: EVM + Privy, not split between active EVM and half-live Solana
- user-facing transaction UX is materially better than before across swap, claim, campaign funding, and vault flows

Still needs cleanup before "100%"

- Supabase migration history is drifted and duplicated
- the canonical architecture docs are stale
- some route/file comments still describe outdated schedules or older system boundaries
- production deployment still needs to be verified from Vercel state, not inferred from git alone

Primary Risks

1. Schema drift

This is the clearest production-readiness risk.

Observed from 'npx supabase migration list':

- duplicate local migration versions:
 - '20260319000001'
 - '20260320000002'
- local-only migrations:
 - '20260329000003'
 - '20260401000004'
 - '20260401000005'
 - '20260401000006'
 - '20260401000007'
- remote-only migrations:
 - '20260401000001'
 - '20260401000002'
 - '20260401000003'

Why it matters:

- repo schema truth is not a clean mirror of production
- future migration application is risky until history is reconciled
- universal rewards and Solana-era tables are especially likely to confuse later work

Recommended next step:

- do not 'db push'
- inspect live schema objects against the repo migrations
- reconstruct the missing April 1 migrations in repo or explicitly document them as production-only history
- normalize duplicate migration numbering before adding more schema work

Additional confirmed live-schema findings:

- 'trade_signals', 'trade_signal_sync_state', 'universal_reward_epochs', and 'universal_reward_allocations' already exist in production
- 'universal_reward_epochs.distribution_id' and 'published_at' also exist in production
- 'sol_distributions' does not exist in production
- 'trade_signals.settled_epoch_id' does not exist in production

Reference:

- ['docs/developers/supabase-migration-reconciliation.md'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/docs/developers/supabase-migration-reconciliation.md)

2. Documentation drift

['docs/ARCHITECTURE.md'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/docs/ARCHITECTURE.md) still reflects a simpler March system. It does not adequately represent:

- Privy as the onboarding layer
- the universal rewards pipeline
- agent APIs and ERC-8004 surfaces
- the current daily Vercel cron schedule
- Solana as paused rather than co-equal

Recommended next step:

- treat ['docs/developers/platform-system-map.md'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/docs/developers/platform-system-map.md) as the current operational map until the canonical architecture doc is rewritten

3. Deploy verification gap

The intended production flow is now clear:

```
'''text
Local repo -> GitHub main -> Vercel production deploy -> mintware.finance
GitHub docs/main -> GitBook sync -> Docs
'''
```

But production truth should still be verified from Vercel, not assumed from pushes.

Recommended next step:

- confirm the current production deployment is serving a commit at or after:
 - '29e3fb62' Solana live pause
 - '0e859b9b' lockfile refresh
 - '92724fae' Ethereum UX upgrade

What Changed Since The Earlier System Map

The platform is no longer just:

- wallet connect
- campaigns
- claims
- one cron rail

It is now:

- Privy + wagmi identity shell
- swap/trading UX and quote proxy
- campaign attribution and claim settlement
- gated vault product with off-chain bookkeeping and rebalancing
- AI agent registration, metadata, and leaderboard
- universal rewards ingestion, settlement, and bridge crons
- public docs pipeline through GitBook sync

That is why a whole-platform map is needed now; the old ?campaign engine plus scoring worker? model is no longer sufficient.

Recommended Next Actions

P0

- reconcile Supabase migration history
- verify current Vercel production deploy commit and health
- rewrite ['docs/ARCHITECTURE.md'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/docs/ARCHITECTURE.md) from the newer platform map

P1

- audit cron comments and stale route headers so code comments match actual schedules
- document which vault routes are intended to be dark when Phase 2 is off
- document which Solana routes remain intentionally paused versus intentionally deleted

P2

- build a dependency inventory for env vars by subsystem
- add a repeatable launch checklist for GitHub, Vercel, Supabase, and GitBook

Bottom Line

Mintware now has enough moving parts that ?production ready? has to mean more than ?the app builds.? The current state is promising: the live product direction is coherent, the largest user-facing UX gaps were improved, and Solana is no longer muddying the active surface. The remaining work is mainly operational rigor:

- one source of schema truth
- one trustworthy system map

- one verified deploy path

Until those are cleaned up, Mintware is close, but not honestly at a 100% production-ready bar.

```
=====
FILE: docs/developers/final-audit-closeout.md
=====
```

Final Audit Closeout

Last updated: 2026-04-05

Owner: Codex review pass

Purpose

This document closes out the intensive Mintware platform audit and hardening pass completed across:

- application routes and auth boundaries
- Supabase-backed mutation paths
- reward settlement and cron flows
- smart contracts and economic/accounting assumptions
- production deployment verification

Use this as the canonical high-level record of:

- what was reviewed
- what was fixed
- what is live on 'main'
- what remains intentionally out of scope or lower priority

Executive Summary

Mintware is in a materially stronger state than it was at the start of this pass.

The dominant bug class was:

- trusting public wallet strings instead of proving wallet control

That pattern affected claim marking, campaign writes, vault writes, referral/connect flows, wallet linking, agent registration, and several related routes. Those issues were fixed and replayed onto 'main'.

The deeper contract and accounting pass also surfaced real correctness issues:

- compounding and claim accounting seams
- fee-routing/accounting mismatches
- epoch progression stalls
- operator-sensitive economic behavior

Those were also fixed and landed on 'main'.

Scope Covered

Application / API layer

- claim generation and claim marking
- campaign join, create, manage, and score refresh
- referral application and first-connect
- wallet linking
- vault create, deposit, withdraw, and rebalance proposal flows
- agent registration and MWP submission
- swap attribution and reward crediting

Data / Supabase layer

- service-role mutation paths
- migration drift
- epoch/distribution persistence assumptions
- live schema vs repo schema reconciliation

Smart contract / protocol layer

- ['contracts/MintwareDistributor.sol'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/contracts/MintwareDistributor.sol)
- ['contracts-v4/src/FeeVault.sol'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/contracts-v4/src/FeeVault.sol)
- ['contracts-v4/src/SocialVault.sol'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/contracts-v4/src/SocialVault.sol)
- ['contracts-v4/src/MWSocialHook.sol'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/contracts-v4/src/MWSocialHook.sol)
- ['contracts-ai/src/AIAttribution.sol'](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/contracts-ai/src/AIAttribution.sol)

Production / deploy layer

- GitHub 'main'
- Vercel production deployment and aliasing
- cron auth behavior
- production deployment recency vs hardened 'main'

Major Fix Buckets

1. Wallet-proof and write-auth hardening

Fixed:

- public claim-mark secret dependence
- unauthenticated claim-mark mutation
- unauthenticated agent registration / MWP writes
- unauthenticated vault create / deposit / withdraw writes
- unauthenticated campaign create / manage writes
- unauthenticated connect / referral activation
- unauthenticated referral apply
- wallet-link without EVM ownership proof

Result:

- important mutation routes now require wallet-signed authorization or verified on-chain transaction proof

2. Reward attribution and settlement hardening

Fixed:

- fail-open swap tx verification
- public swap-event ingestion crediting unverifiable txs
- token-pool pending distributions with no real publish retry
- empty / zero-point epoch stalls
- claim accounting mismatch between 'MintwareDistributor' and 'FeeVault'
- stale DB mirror dependence for vault epoch pool totals

Result:

- reward settlement now fails closed more often, retries where appropriate, and keeps contract/off-chain accounting better aligned

3. Smart contract hardening

Fixed:

- 'SocialVault' first-caller seeding risk
- lock-tier downgrade / lock rewrite via small top-up
- stale ERC-8004 reverse links in 'AIAtribution'
- 'FeeVault.compound()' pretending to work while not actually compounding
- invalid pool/config guard gaps
- compounding enabled in contracts before the off-chain pipeline could safely support it
- fee share changes retroactively affecting the current epoch

Result:

- core contracts now preserve the intended architecture with tighter correctness and safer operator behavior

4. MEV capture and fee normalization

Fixed:

- non-USDC captured value being mis-accounted as USDC
- direct fee inflows bypassing 'FeeVault' epoch accounting

Shipped design:

- USDC capture goes directly into 'FeeVault'
- non-USDC capture stages to treasury
- staged assets can be normalized into USDC and then deposited into 'FeeVault'

Result:

- Mintware is not capped to earning only in USDC at the hook layer, while accounting remains USDC-normalized for the current fee-distribution system

5. Economic / incentive corrections

Fixed:

- referral squatting via public referred-wallet submission

- referral-trade points using the trader's multiplier instead of the referrer's
- early epoch close and other operator-sensitive epoch timing issues

Result:

- incentives are more aligned to the actual wallet earning them, and operators have less room to retroactively reshape live economics

Production Status

Current branch truth

- 'origin/main' includes the application hardening, contract hardening, accounting fixes, and economic fixes from this audit pass

Production truth

Verified:

- <https://mintware.finance>
- <https://www.mintware.finance>

Vercel production deployment:

- 'dpl_8PpR18Qw765uaRxJHKK9dnVxzb4t'
- target: 'production'
- status: 'Ready'

At verification time, production timing matched the latest hardened 'main' commit window closely enough to treat the live deployment as caught up to the hardening pass.

Remaining Open Items

Not currently treated as critical

- some stale comments and low-severity docs drift may still exist outside the touched paths
- full manual browser-based live wallet execution was not performed for every user flow in this session
- Foundry full 'forge test' remained flaky on this machine because of local environment/proxy issues, although compile-level verification succeeded

Still worth doing

- add more invariant / fuzz coverage around the audit-boundary contracts
- keep cron and settlement monitoring tight
- do one deliberate manual production smoke round with real wallets

What This Is Not

This was a serious full-platform review pass, but it is not the same thing as:

- a paid third-party formal smart-contract audit
- a proof that zero bugs remain
- a guarantee that future changes can ignore the same rigor

The value of this pass is that it removed a large set of real, production-relevant flaws

and converted Mintware from a drift-heavy system into one with a much stronger operational and security baseline.

Recommended Next Steps

Immediate

- keep this document and the operator checklist current as the post-audit source of truth
- run one manual live-wallet smoke test across the highest-value flows

Short term

- add invariant/fuzz tests for the audit-boundary contracts
- narrow and freeze the external-audit boundary before paying for outside review

Product track after that

- 'Privy' embedded-wallet gas-with-token MVP

Bottom Line

Mintware is in a much stronger position than when this pass started.

The highest-risk off-chain auth problems, reward-settlement seams, fee-accounting issues, and core economic correctness bugs found in this review were fixed and landed on 'main'.

The platform is now much closer to:

- safe enough to operate
- clear enough to hand off
- and disciplined enough to keep iterating without pretending the hard parts are solved forever

=====

FILE: docs/developers/production-operator-checklist.md

=====

Production Operator Checklist

Last updated: 2026-04-05

Owner: Codex review pass

Purpose

This is the practical operator-side checklist for running Mintware safely after the audit hardening pass.

Use this before:

- deploying new production code
- enabling new reward rails
- operating vault and epoch settlement
- handing the platform to another operator or reviewer

Core Assumptions

Mintware currently assumes:

- EVM is the live transaction rail
- Solana live surfaces remain paused
- 'FeeVault' accounting is USDC-normalized
- non-USDC MEV capture stages to treasury, then normalizes into USDC
- important write paths require wallet proof or on-chain verification
- cron routes fail closed outside local development when 'CRON_SECRET' is missing

Before Deploying

- confirm the target branch is 'main'
- confirm 'origin/main' contains the latest hardening commits
- confirm no security-sensitive work is stranded only on 'codex/*' branches
- confirm local-only files are not being committed:
 - [/.claude/launch.json](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/.claude/launch.json)
 - [/.claude/rules/deployments.md](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/.claude/rules/deployments.md)
 - [/supabase/.temp/cli-latest](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/supabase/.temp/cli-latest)
 - [/docs/.DS_Store](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/docs/.DS_Store)

Vercel Checklist

- verify latest production deploy is 'Ready'
- verify aliases point to the latest production deploy:
 - 'mintware.finance'
 - 'www.mintware.finance'
- confirm the live production deploy is on or after the expected 'main' commit
- confirm cron routes in 'vercel.json' still match the intended live schedule

Supabase Checklist

- confirm project ref is the intended live project
- confirm migration history is clean enough for the intended change
- never run blind 'db push' against production
- for schema changes:
 - prove live schema markers first
 - reconcile migration history deliberately
 - only then apply unapplied migrations

Contract / Treasury Checklist

- confirm 'FeeVault' treasury address is correct
- confirm 'FeeVault' oracle signer is correct
- confirm 'SocialVault' / 'MWSocialHook' addresses are wired as expected
- confirm 'mevTreasury' staging address is correct
- confirm fee-share changes are intentional for future epochs only
- remember:
 - current epoch shares are now snapshotted at epoch open
 - 'compound()' is intentionally disabled

Cron / Settlement Checklist

- 'epoch-end'
 - expired points epochs should advance even when empty

- 'pool-settle'
 - claimable token-pool rewards should settle into distributions
 - pending token-pool roots should retry publication
- 'vault-epoch-close'
 - on-chain 'FeeVault' totals should be available when configured
- 'treasury/normalize-mev'
 - non-USDC staged assets should normalize into USDC and deposit into 'FeeVault'
- 'universal-pipeline'
 - if enabled, should fail closed when required env is missing

Live Smoke Test Checklist

Identity / referral

- connect wallet returns a ref code
- referral apply requires wallet consent
- connect activates only the connecting wallet's pending referral state

Campaigns

- join works for eligible wallets
- create/manage requires creator wallet proof
- score refresh does not break participant state

Claims

- claim status loads
- claim proof route returns distribution data for eligible wallets
- mark-claimed only succeeds on matching distributor calldata

Vaults

- create requires signed auth
- deposit requires signed auth and verified deposit tx
- withdraw requires signed auth and verified withdrawal tx
- rebalance proposal submission only accepts matching on-chain proposal executions

Rewards / settlement

- swap-event rejects unverifiable txs
- token-pool pending distributions eventually become published
- points epochs advance cleanly

Operational Rules

- do not assume a cron comment is true unless the route behavior and 'vercel.json' agree
- do not assume a database mirror is authoritative if an on-chain source of truth exists
- do not assume a wallet address in a request body proves ownership
- do not enable half-wired economics in production just because the contract surface exists

External Audit Boundary

If paying for a focused external review later, scope the money-bearing core first:

- [contracts/MintwareDistributor.sol](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/contracts/MintwareDistributor.sol)

```
- [contracts-  
v4/src/FeeVault.sol](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Build/c  
ontracts-v4/src/FeeVault.sol)  
- [contracts-  
v4/src/SocialVault.sol](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Bui  
ld/contracts-v4/src/SocialVault.sol)  
- [contracts-  
v4/src/MWSocialHook.sol](/Users/nicolasrobinson/Downloads/Mintware%20Phase%201%20app%20Bui  
ld/contracts-v4/src/MWSocialHook.sol)  
- [contracts-ai/src/AIAttribution.sol](/Users/nicolasrobinson/Downloads/Mintware%20Phase%2  
01%20app%20Build/contracts-ai/src/AIAttribution.sol)
```

Everything else should support that boundary, not expand it casually.

Bottom Line

Mintware is in a stronger place now, but the way it stays that way is simple:

- verify live truth
- keep assumptions explicit
- keep writes authenticated or chain-verified
- keep accounting tied to the real source of truth
- and do not reintroduce convenience shortcuts in production paths

```
=====
```

FILE: docs/developers/supabase-migration-reconciliation.md

```
=====
```

Supabase Migration Reconciliation

Last audited: 2026-04-03
Owner: Codex review pass

Purpose

This document records the current drift between repo migration history and the linked live Supabase project, plus the safest next actions.

It is intentionally non-destructive. Nothing here assumes 'db push', force-repair, or history rewriting without review.

Linked Project

- project ref: 'bqwcwrnqpayfndgmceal'
- project name: 'Mintware Project'
- project status: 'ACTIVE_HEALTHY'

History Drift Summary

From 'npx supabase migration list':

Duplicate local version numbers

- '20260319000001'
- '20260320000002'

These duplicate-version files are already a repo hygiene problem even before considering

production drift.

Local-only migration versions

- '20260329000003'
- '20260401000004'
- '20260401000005'
- '20260401000006'
- '20260401000007'

Remote-only migration versions

- '20260401000001'
- '20260401000002'
- '20260401000003'

What The Live Schema Proves

Direct linked-database inspection shows:

Present in production

- 'trade_signals'
- 'trade_signal_sync_state'
- 'universal_reward_epochs'
- 'universal_reward_allocations'
- 'universal_reward_epochs.distribution_id'
- 'universal_reward_epochs.published_at'
- indexes:
 - 'trade_signals_chain_tx_log_uidx'
 - 'trade_signals_pool_block_idx'
 - 'trade_signals_swapper_idx'
 - 'universal_reward_allocations_epoch_idx'
 - 'universal_reward_allocations_recipient_idx'
 - 'universal_reward_epochs_distribution_uidx'
 - 'universal_reward_epochs_pool_epoch_idx'

Not present in production

- 'sol_distributions'
- 'trade_signals.settled_epoch_id'

RLS policy state

No policies were found for:

- 'trade_signals'
- 'trade_signal_sync_state'
- 'universal_reward_epochs'
- 'universal_reward_allocations'

That does not automatically mean the tables are unsafe, because service-role access may be the only intended path, but it should be treated as an explicit design choice rather than an accidental default.

Interpretation

'20260329000003_sol_distributions.sql'

This migration is local-only and its main table is not present in production.

Practical meaning:

- the Solana distributor rail is not active in the live database
- this is consistent with Solana now being paused from the live product surface

'20260401000004' through '20260401000007'

These versions are local-only in history, but most of their schema effects are already present remotely.

Practical meaning:

- production likely received equivalent changes through the unknown remote-only April 1 migrations
- migration history and schema reality have diverged
- the bridge columns landed partially:
 - 'distribution_id' and 'published_at' exist
 - 'settled_epoch_id' does not

'20260401000001' through '20260401000003'

These are remote-only in history, but they almost certainly correspond to the universal-rewards schema that production already has.

Practical meaning:

- we are missing source-of-truth migration files in the repo for at least part of the universal rollout
- the repo should not pretend it fully represents production schema history today

Safe Next Actions

P0

- do not run 'npx supabase db push'
- do not create more migrations on top of this drift until the numbering/history issue is addressed
- preserve the current linked project and history as evidence

P1

- reconstruct or recover the missing remote-only migration files:
 - '20260401000001'
 - '20260401000002'
 - '20260401000003'
- determine whether 'trade_signals.settled_epoch_id' is still required by the current universal pipeline

P2

- decide whether 'sol_distributions' should:
 - remain local-only and dormant, or
 - be removed from the active migration path entirely while Solana is paused

P3

- normalize duplicate version numbers in the repo before future schema work
- add explicit documentation on whether the universal tables are intentionally service-role-only or should have RLS policies

Bottom Line

The live database is healthier than the migration list alone suggests, because the universal rewards schema is already materially present. But the repo cannot currently claim to be a clean canonical record of production schema history.

The most accurate statement is:

- production schema contains most of the universal pipeline tables and bridge fields
- repo migration history is missing part of how production got there
- Solana distributor schema is not live
- future migration work should be treated as reconciliation work first, feature work second